

ASP. NET Web API 2.2



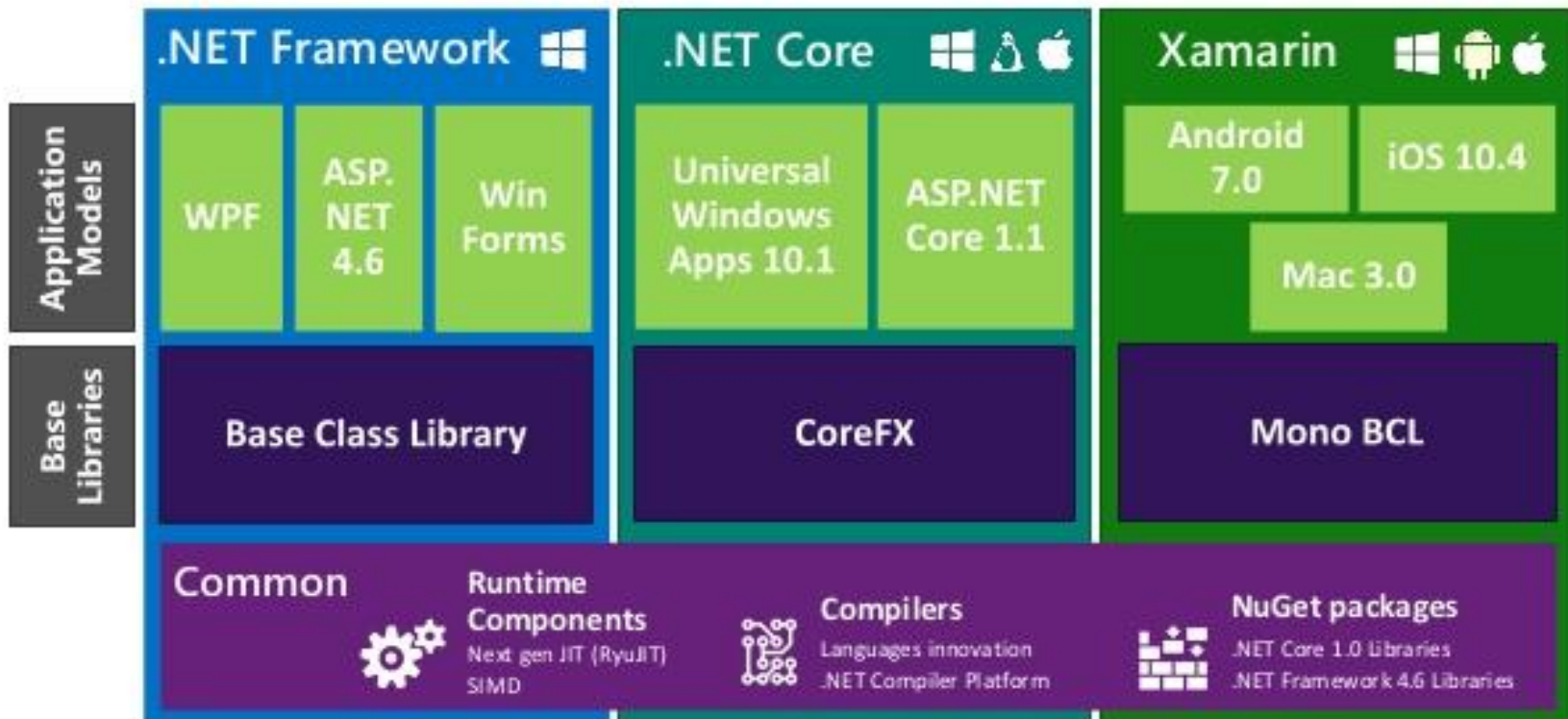
Agenda

- Big Picture
- SOAP? REST? OData?
- Web API Framework
- Hosting
- Security
- Consuming
- Clients mit JavaScript und C#

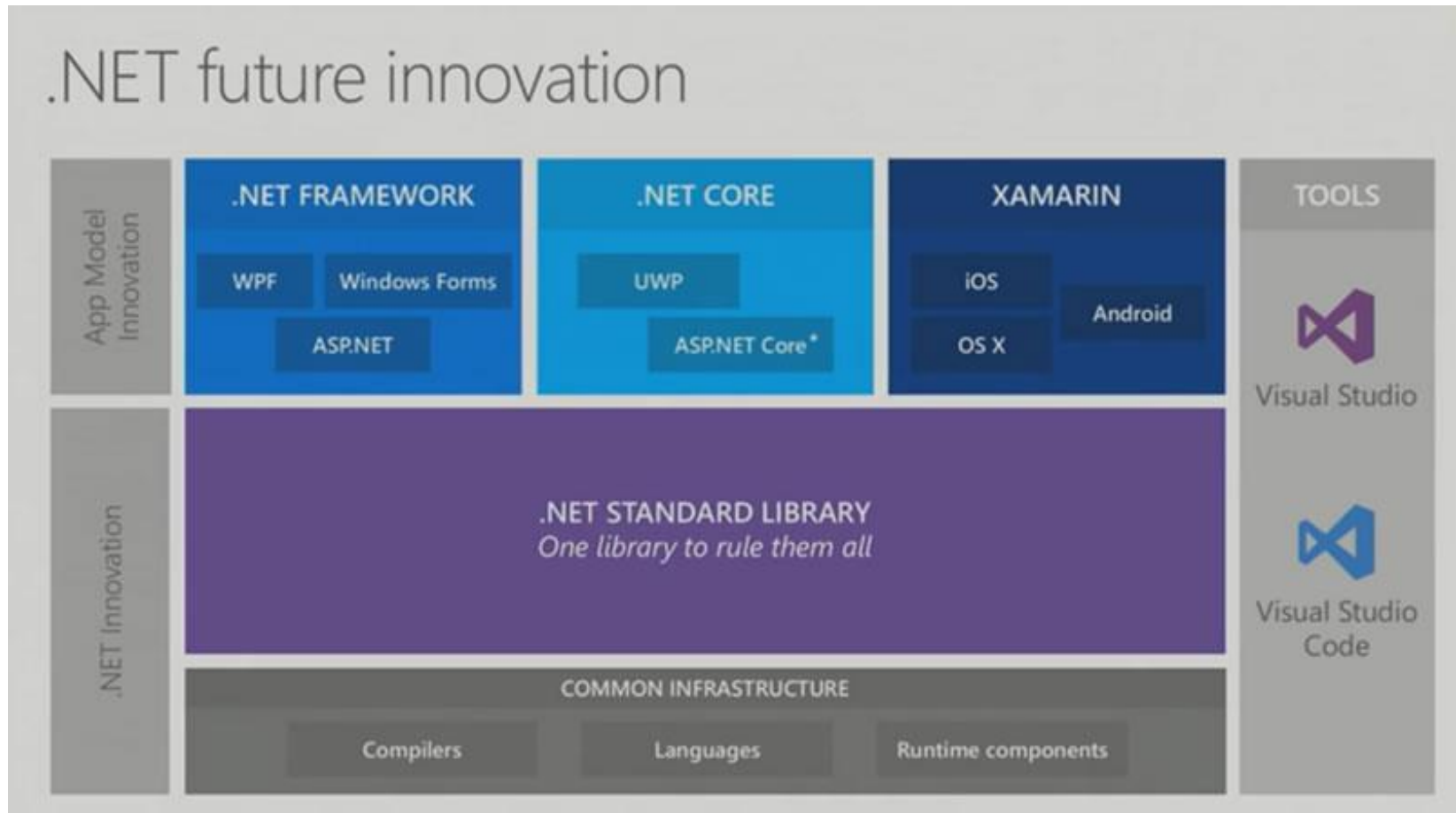
.NET Übersicht

The big picture of .NET Platforms

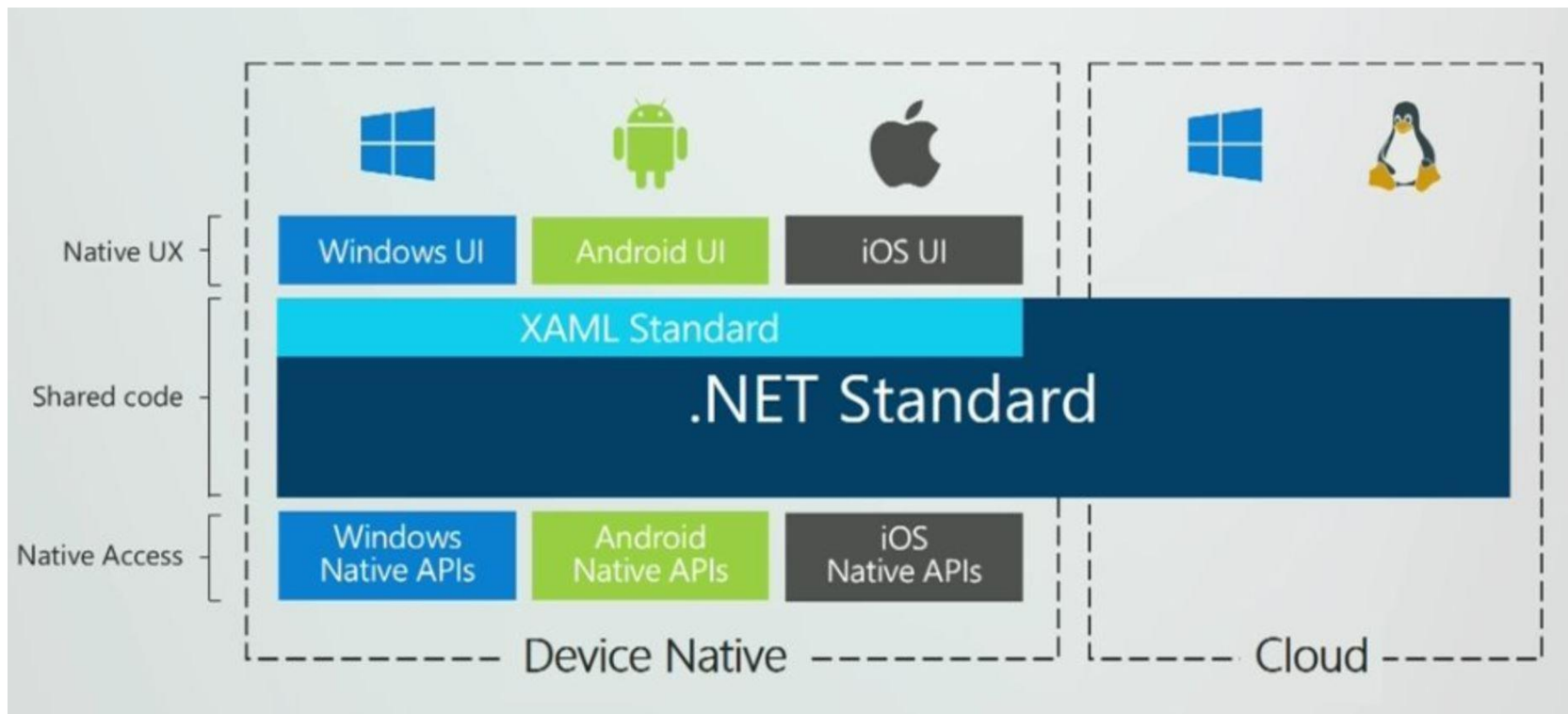
DOT
NEXT



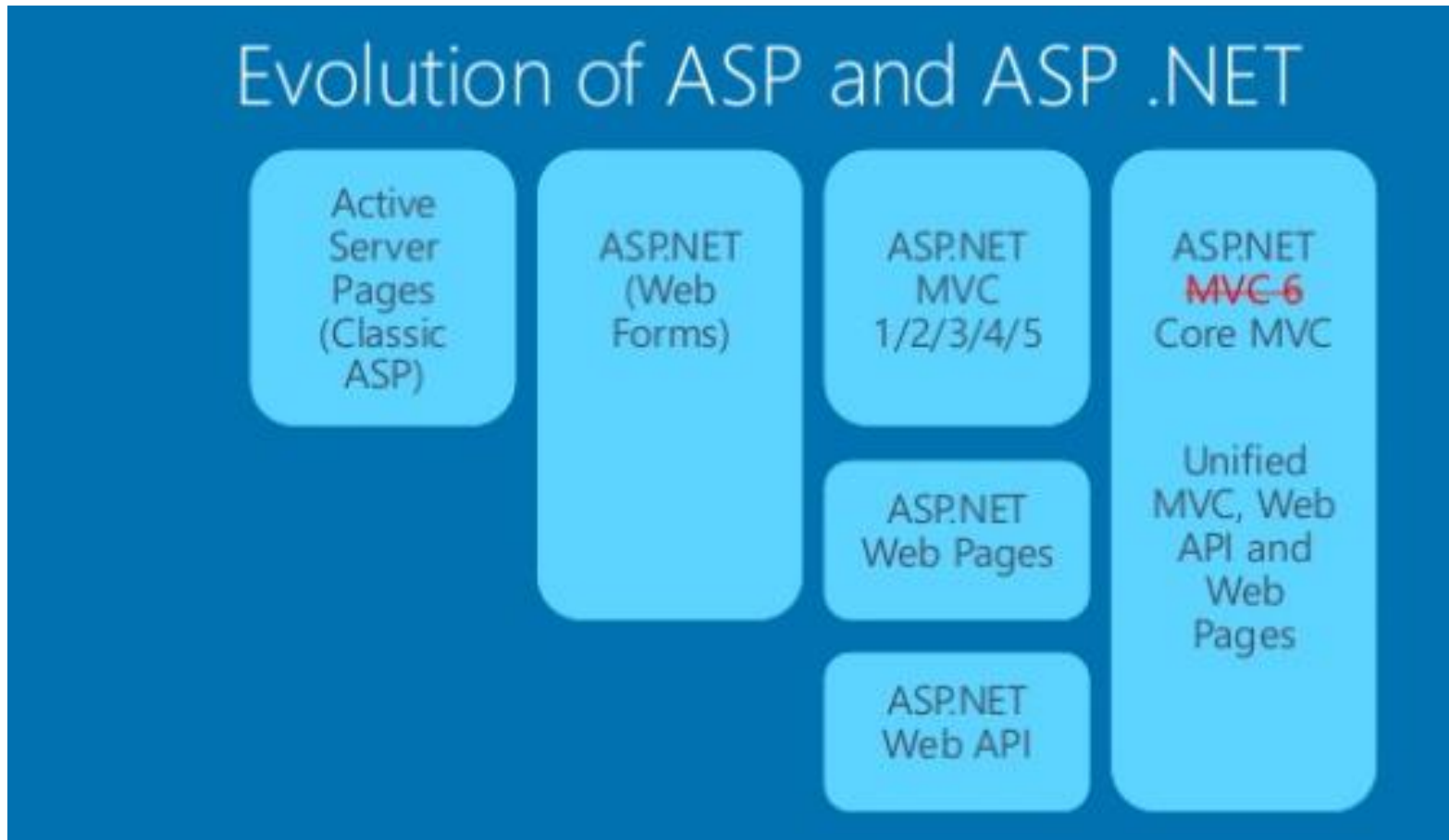
.NET Übersicht



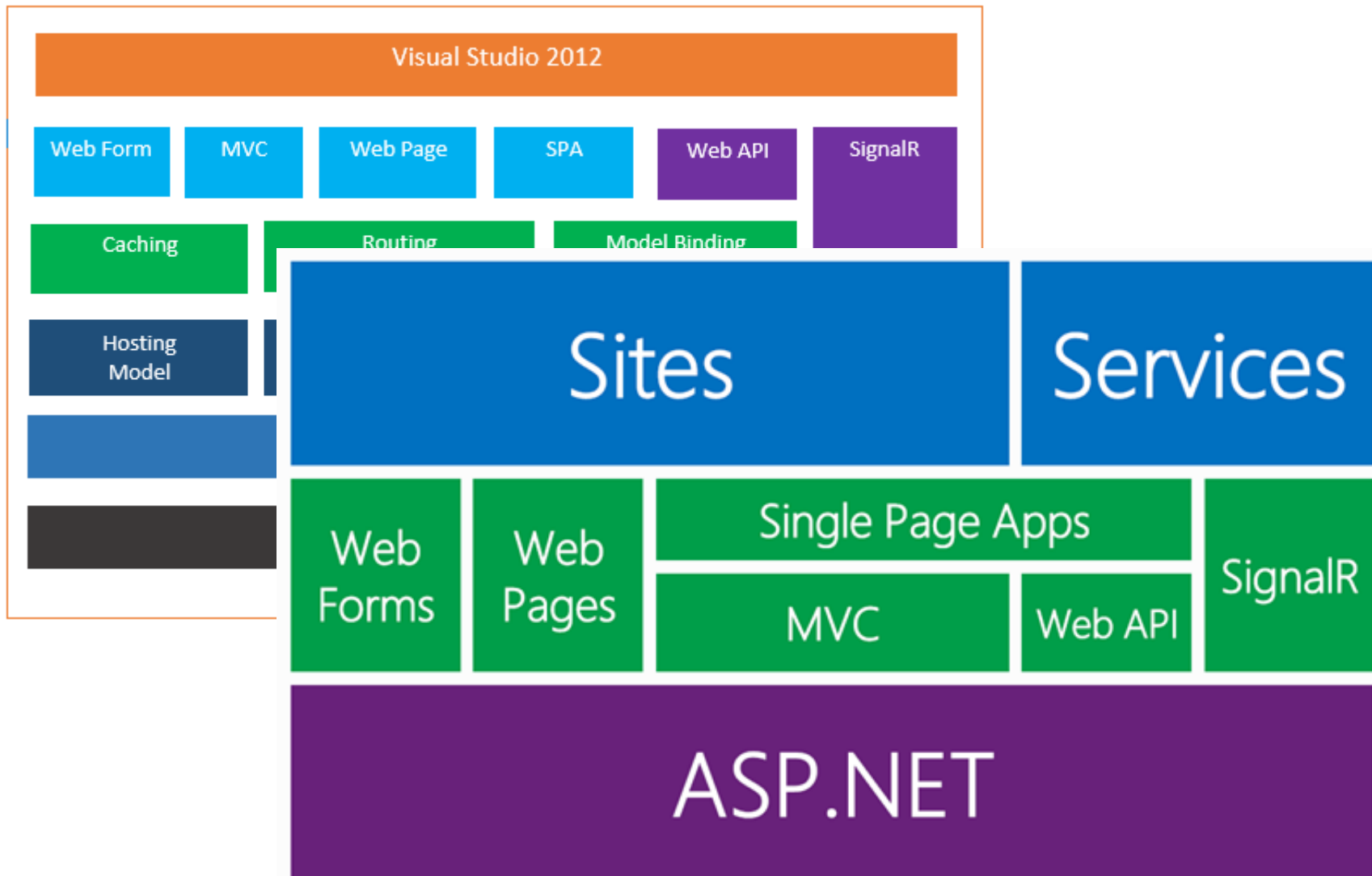
.NET Übersicht



ASP -> ASP.NET



ASP.NET 4 / 5 , oder wie ?



ASP.NET 4 / 5 , oder wie ?

It's very easy to armchair quarterback and say that "they should have named it Foo and it would be easy" but very often there's many players involved in naming things. ASP.NET is a good 'brand' that's been around for 15 years or so. ASP.NET 4.6 is a supported and released product that you can get and use now from <http://get.asp.net>.

However, naming the new, completely written from scratch ASP.NET framework "ASP.NET 5" was a bad idea for a one major reasons: 5 > 4.6 makes it seem like ASP.NET 5 is bigger, better, and replaces ASP.NET 4.6. Not so.

So we're changing the name.

Reintroducing ASP.NET Core 1.0 and .NET Core 1.0

- ASP.NET 5 is now ASP.NET Core 1.0.
- .NET Core 5 is now .NET Core 1.0.
- Entity Framework 7 is now Entity Framework Core 1.0 or EF Core 1.0 colloquially.

Why 1.0? Because these are new. The whole .NET Core concept is new. The .NET CLI is very new. Not only that, but .NET Core isn't as complete as the full .NET Framework 4.6. We're still exploring server-side graphics libraries. We're still exploring gaps between ASP.NET 4.6 and ASP.NET Core 1.0.

Quelle: <https://blogs.msdn.microsoft.com/webdev/2016/01/19/asp-net-5-is-dead-introducing-asp-net-core-1-0-and-net-core-1-0/>

ASP.NET 4 / 5 , oder wie ?

ASP.NET 4.6 and ASP.NET Core 1.0

ASP.NET 4.6	ASP.NET Core 1.0
.NET Framework 4.6 	.NET Core 1.0   
.NET framework libraries	.NET core libraries
Compilers and runtime components (.NET Compiler Platform: Roslyn, C#, VB, F# Languages, RyuJIT, SIMD)	

To be clear, ASP.NET 4.6 is the more mature platform. It's battle-tested and released and available today. ASP.NET Core 1.0 is a 1.0 release that includes Web API and MVC but doesn't yet have SignalR or Web Pages. It doesn't yet support VB or F#. It will have these subsystems some day but not today.

Quelle: <https://blogs.msdn.microsoft.com/webdev/2016/01/19/asp-net-5-is-dead-introducing-asp-net-core-1-0-and-net-core-1-0/>

Microsofts WebTechnologien

	Web Forms	MVC	Single Page Application	Web Sites	Dynamic Data	ASP.NET Core	Web API	WCF Services
Betriebs-system	Windows	Windows	Windows	Windows	Windows	Windows, Mac, Linux	Windows	Windows
vorgestellt mit	.NET 1.0	.NET 3.5	.NET 4	.NET 1.1	.NET 2.0	.NET Core 1.0	.NET 3.5	.NET 2.0
Framework	.NET 4.7	.NET 4.7	.NET 4.7	.NET 4.7	.NET 4.7	.NET 4.7 / .NET Core 1.0	.NET 4.7	.NET 4.7
Plattformen	.NET Framework	.NET Framework	.NET Framework	.NET Framework	.NET Framework	.NET Framework / .NET Core	.NET Framework / .NET Core	.NET Framework
Verwendungs-zweck	Web-UI	Web-UI	Web-UI	Web-UI	Web-UI	Web-UI / Data Services	Data Services	Data Services
Anwen-dungstyp	Web-anwendung	Website, Web-anwendung	Web-anwendung	Website	Backend	Website, Web-anwendung	Datendienste	Datendienste
Projekttyp	UI- und daten-orientiert	UI-orientiert	UI-orientiert	UI-orientiert	daten-orientiert	UI-orientiert	schnitt-stellen- und daten-orientiert	schnitt-stellen- und daten-orientiert
App-Größe	*****	*****	*	**	*	*****	***	***
Einstieg	****	***	**	****	**	***	***	***
Customizing	****	****	****	**	***	****	****	**
Dokumen-tation	*****	*****	**	***	***	***	*****	**
Community	*****	*****	**	*	*	****	****	*

LINQ

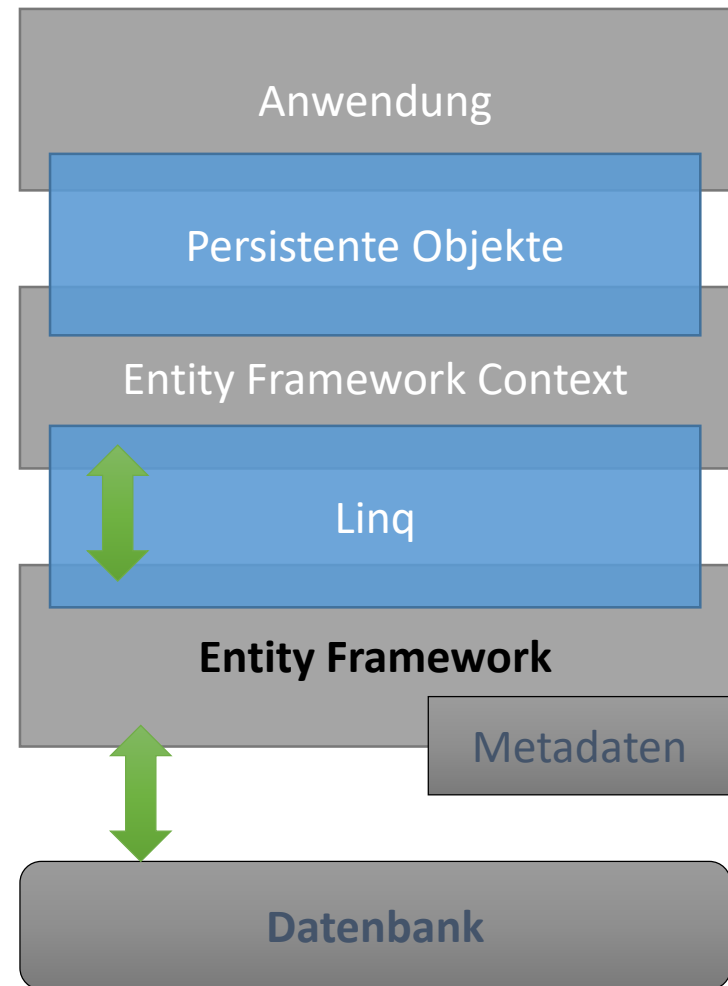
- **Exkurs LINQ**

- Language Integrated Query
- Abfragesprachen auf Objektmengen
- Direkter Bestandteil von C# und VB.NET
- Bietet die Möglichkeit, Abfragen abstrakt zu beschreiben, die später durch entsprechende Provider in konkrete Abfragen für die entsprechenden Zielsysteme überführt werden.
- 101 LINQ Beispiele:
 - <https://code.msdn.microsoft.com/101-LINQ-Samples-3fb9811b>

Entity Framework

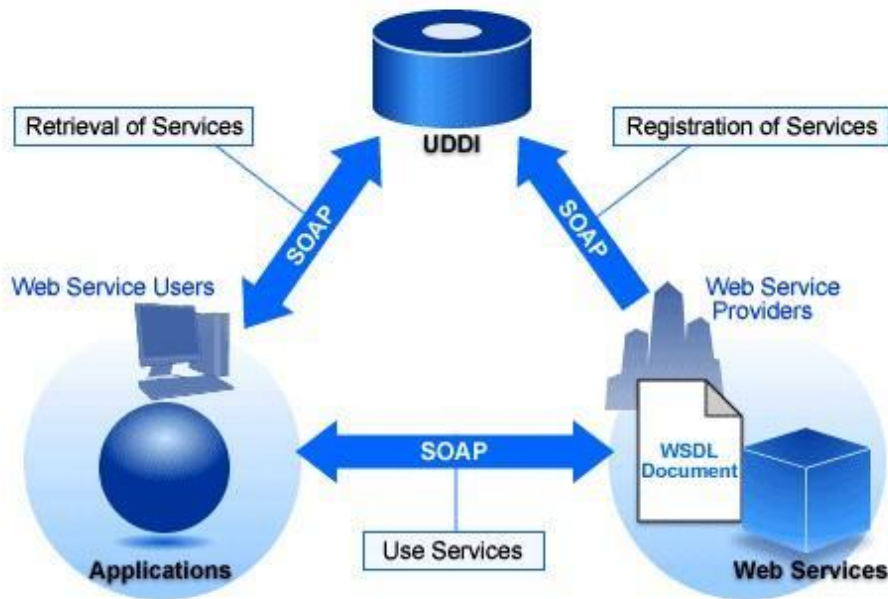
- **Exkurs Entity Framework**

- O/R Mapping Framework
- Übersetzt LINQ in SQL
- Abstrahiert ADO.NET
- Ermöglicht es, Daten nur bei Bedarf zu laden (*lazy loading*)
- Der Einsatz von O/R Mappern spart viel Zeit!



Was ist ein Webdienst ?

Ein kurzer Rückblick.



Was ist ein Webdienst ?

Ein kurzer Rückblick.

Metadata Specifications

WS-Policy

1.1
W3C
Working Draft

▲ WS-Policy describes the capabilities and constraints of the policies on intermediaries and endpoints (e.g. business rules, required security tokens, supported encryption algorithm, policy rules).

WS-PolicyAssertions

1.1
BEA Systems, IBM, Microsoft, SAP
Public Draft

▲ WS-PolicyAssertions provides an initial set of assertions to address some common needs of Web Services applications.

WS-PolicyAttachment

1.2
W3C
W3C Member Submission

▲ WS-PolicyAttachment defines two general-purpose mechanisms for associating policies with the subjects to which they apply; the policies may be defined as part of existing metadata about the subject or the policies may be defined independently and associated through an external binding to the subject.

WS-Discovery

Microsoft, BEA Systems, Canon, Intel and webMethods
Draft

▲ WS-Discovery defines a multicast discovery protocol for dynamic discovery of services on ad-hoc and managed networks.

WS-MetadataExchange

1.1
BEA Systems, Computer Associates, IBM, Microsoft, SAP, Sun Microsystems and webMethods
Public Draft

▲ WS-MetadataExchange enables a service to provide metadata to others through a Web Services interface. One who references a Web Service, a user can access a set of WS-Exchange operations to retrieve the metadata that describes the service.

Universal Description, Discovery and Integration (UDDI)

3.0.2
OASIS
OASIS-Standard

▲ Universal Description, Discovery and Integration (UDDI) defines a set of services supporting the description and discovery of business, organizations, and other Web services providers. The Web services they make available, and the technical interfaces which may be used to access these services.

Web Service Description Language 2.0 SOAP Binding

2.0
W3C - Working Draft

▲ Web Service Description Language SOAP Binding describes the concrete details for using WSDL 2.0 in conjunction with SOAP 1.1 protocol.

Web Service Description Language 2.0 Core

2.0
W3C
Candidate Recommendation

▲ Web Service Description Language 2.0 Core is an XML-based language for describing Web services and how to access them. It specifies the location of the service and the operations for methods the service exposes.

Web Service Description Language 1.1

1.1
W3C
Note

▲ Web Service Description Language 1.1 is an XML-based language for describing Web services and how to access them. It specifies the location of the service and the operations for methods the service exposes.

Reliability Specifications

WS-ReliableMessaging

1.1
OASIS
Committee Draft

▲ WS-ReliableMessaging describes a protocol that allows Web services to communicate reliably in the presence of software component, system, or network failures. It defines a SOAP binding that is required for interoperability.

WS-Reliable Messaging Policy Assertion (WS-AM Policy)

1.1
OASIS
Committee Draft

▲ Web Services Reliable Messaging Policy Assertion (WS-AM Policy) describes a domain-specific policy assertion for WS-ReliableMessaging that can be specified within a policy alternative as defined in WS-Policy framework.

WS-Reliability

1.1
OASIS
OASIS-Standard

▲ WS-Reliability is a SOAP-based protocol for exchanging SOAP messages with guaranteed delivery, no duplicates, and guaranteed message ordering. WS-Reliability is defined as SOAP message extensions and is independent of the underlying protocol. This specification contains a binding to HTTP.

Security Specifications

WS-Security

1.1
OASIS
OASIS-Standard

▲ WS-Security is a communications protocol providing a means for applying security to Web Services.

WS-SecurityPolicy

1.1
IBM, Microsoft, RSA Security, VeriSign
Public Draft

▲ WS-SecurityPolicy defines how to describe policies related to various features defined in the WS-Security specifications.

WS-Security: SOAP Message Security

1.1
OASIS
Public Review Draft

▲ WS-Security: SOAP Message Security describes enhancements to SOAP messaging to provide message integrity and confidentiality. Specifically, this specification provides support for multiple security token formats, trust domains, signature formats and encryption technologies. The token formats and semantics for using these are defined in the associated profile documents.

WS-Security: Username Token Profile

1.1
OASIS
Public Review Draft

▲ WS-Security: Username Token Profile describes how a Web Service consumer can supply a Username Token as a means of identifying the requester by username, and optionally using a password (or shared secret, etc.) to authenticate that identity to the Web Service providers.

WS-Security: Kerberos Binding

1.0
Microsoft, IBM, OASIS
Working Draft

▲ WS-Security: Kerberos Binding defines how to encode Kerberos tickets and attach them to SOAP messages. In well, it specifies how to add signatures and encryption to the SOAP message, in accordance with WS-Security, which uses and references the Kerberos tickets.

WS-Federation

1.0
IBM, Microsoft, BEA Systems, RSA Security, and VeriSign
Initial Draft

▲ WS-Federation describes how to manage and broker the trust relationships in a heterogeneous federated environment including support for federated identities.

WS-Security: SAML Token Profile

1.1
OASIS
Public Review Draft

▲ WS-Security: SAML Token Profile defines the use of Security Assertion Markup Language (SAML) v1.1 assertions in the context of WS-Security SOAP Message Security including for the purpose of securing SOAP messages and SOAP message exchanges.

WS-Trust

BEA Systems, Computer Associates, IBM, Layer 7 Technologies, Microsoft, Netegrity, Otilo, OpenNetwork, Ping Identity Corporation, Reactive, RSA Security, VeriSign and WebMethods
Technology - Initial Draft

▲ WS-Trust describes a framework for trust models that enables Web Services to securely interact. It uses WS-Security base mechanisms and defines additional primitives and extensions for security token exchange to enable the issuance and dissemination of contextual, verifiable trust domain.

WS-Security: X.509 Certificate Token Profile

1.1
OASIS
Public Review Draft

▲ WS-Security: X.509 Certificate Token Profile describes the use of the X.509 authentication framework with the WS-Security SOAP Message Security specification.

WS-SecureConversation

BEA Systems, Computer Associates, IBM, Layer 7 Technologies, Microsoft, Netegrity, Otilo, OpenNetwork, Ping Identity Corporation, Reactive, RSA Security, VeriSign and WebMethods
Technology - Public Draft

▲ WS-SecureConversation specifies how to manage and authenticate message exchanges between parties including security context exchange and establishing and deriving session keys.

Transaction Specifications

WS-Coordination

1.1
OASIS
Working Draft

▲ WS-Coordination describes an extensible framework for providing protocols that coordinate the actions of distributed applications.

WS-Business Activity

1.1
OASIS
Working Draft

▲ WS-Business Activity provides the definition of the business activity coordination type that is to be used with the extensible coordination framework described in the WS-Coordination specification.

WS-Atomic Transaction

1.1
OASIS
Committee Draft

▲ WS-Atomic Transaction defines protocols that enable existing transaction processing systems to keep their proprietary protocols and integrate across different hardware and software vendors.

WS-Composite Application Framework (WS-CAF)

1.0
Ajuna Technologies, Fujitsu, IONA, Oracle and Sun Microsystems
Committee Specification

▲ WS-Composite Application Framework (WS-CAF) is a collection of these specifications aimed at solving problems that arise when multiple Web Services are used in combination. It proposes standard, interoperable mechanisms for managing shared context and ensuring business processes achieve predictable results and recovery from failure.

WS-Context (WS-CTX)

1.0
Ajuna Technologies, Fujitsu, IONA, Oracle and Sun Microsystems
Committee Draft

▲ WS-Context (WS-CTX) is intended as a lightweight mechanism for allowing multiple Web Services to share a common context.

WS-Coordination Framework (WS-CF)

1.0
Ajuna Technologies, Fujitsu, IONA, Oracle and Sun Microsystems
Committee Draft

▲ WS-Coordination Framework (WS-CF) allows the management and coordination in a Web Services interaction of a number of activities related to an overall application.

WS-Transaction Management (WS-TM)

1.0
Ajuna Technologies, Fujitsu, IONA, Oracle and Sun Microsystems
Committee Draft

▲ WS-Transaction Management (WS-TM) defines a core infrastructure service consisting of a Transaction Service for Web Services.

Resource Specifications

Web Services Resource Framework (WSRF)

1.2
OASIS
OASIS-Standard

▲ Web Services Resource Framework (WSRF) defines a family of specifications for accessing stateful resources using Web Services.

WS-BaseFaults (WSBF)

1.2
OASIS
Working Draft

▲ WS-BaseFaults (WSBF) defines a base set of information that may appear in fault messages. WS-BaseFaults defines an XML schema type for base faults, along with rules for how this base fault type is used and extended by Web Services.

WS-ServiceGroup (WSRG)

1.2
OASIS
Working Draft

▲ WS-ServiceGroup (WSRG) defines a means by which Web Services and WS-Resources can be aggregated or grouped together for a domain specific purpose.

WS-ResourceProperties

1.2
OASIS
Working Draft

▲ WS-ResourceProperties specifies the means by which the definition of the properties of a WS-Resource may be described as part of the Web Service interface. The description of the WS-Resource properties represents a projection of a view on the WS-Resource state.

WS-ResourceLifetime

1.2
OASIS
Working Draft

▲ WS-ResourceLifetime is to standardize the terminology, concepts, message exchanges, WSDL and XML needed to manage the lifetime of, and destroy WS-Resources. Additionally, it defines resource properties that may be used to request and monitor the lifetime of a WS-Resource.

WS-Transfer

W3C
W3C Member Submission

▲ WS-Transfer describes a general SOAP-based protocol for accessing XML representations of Web service-based resources.

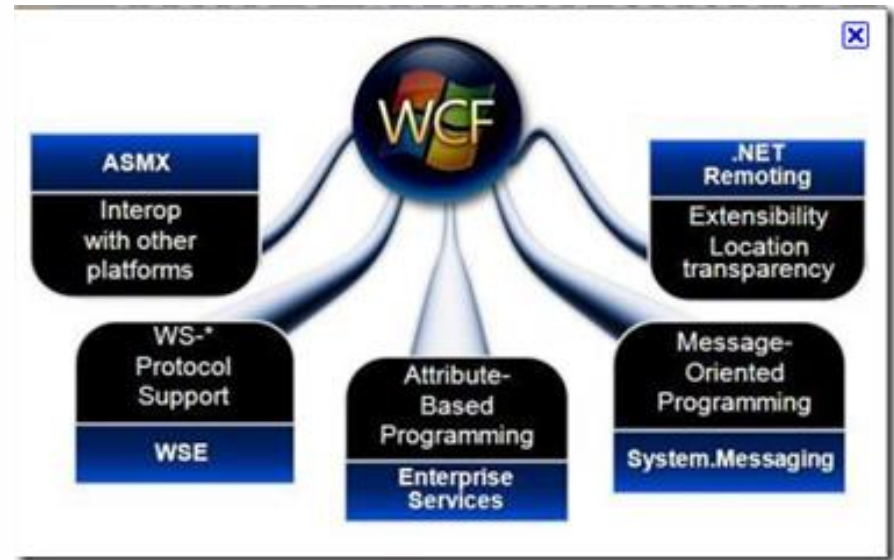
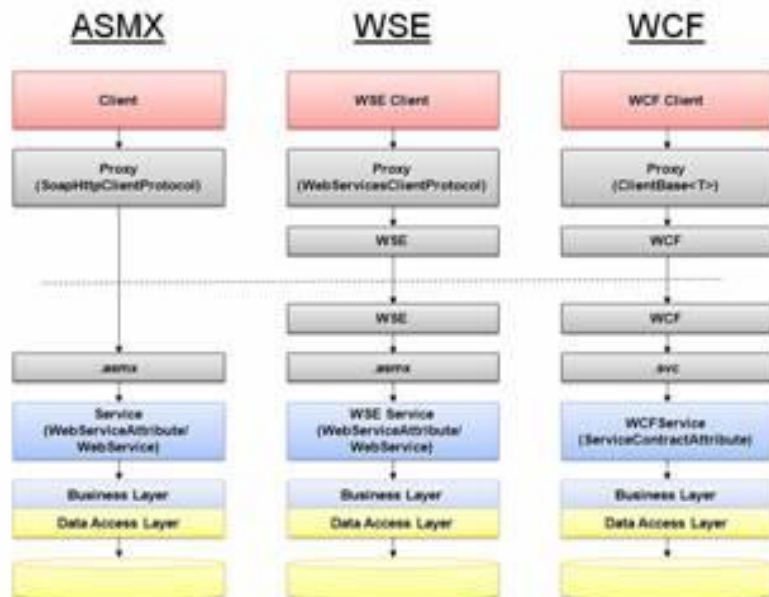
Resource Representation SOAP Header Block (RRSHB)

W3C
Recommendation

▲ Resource Representation SOAP Header Block (RRSHB) complements RRSHB by defining mechanisms for describing and conveying non-XML resource representations in a SOAP 1.2 message.

Was ist ein Webdienst ?

Ein kurzer Rückblick.



Demo

SOA ?



SOA is an architectural style designed for building distributed



The following properties define a service-oriented architecture:



Services handle data, not objects



Services are autonomous



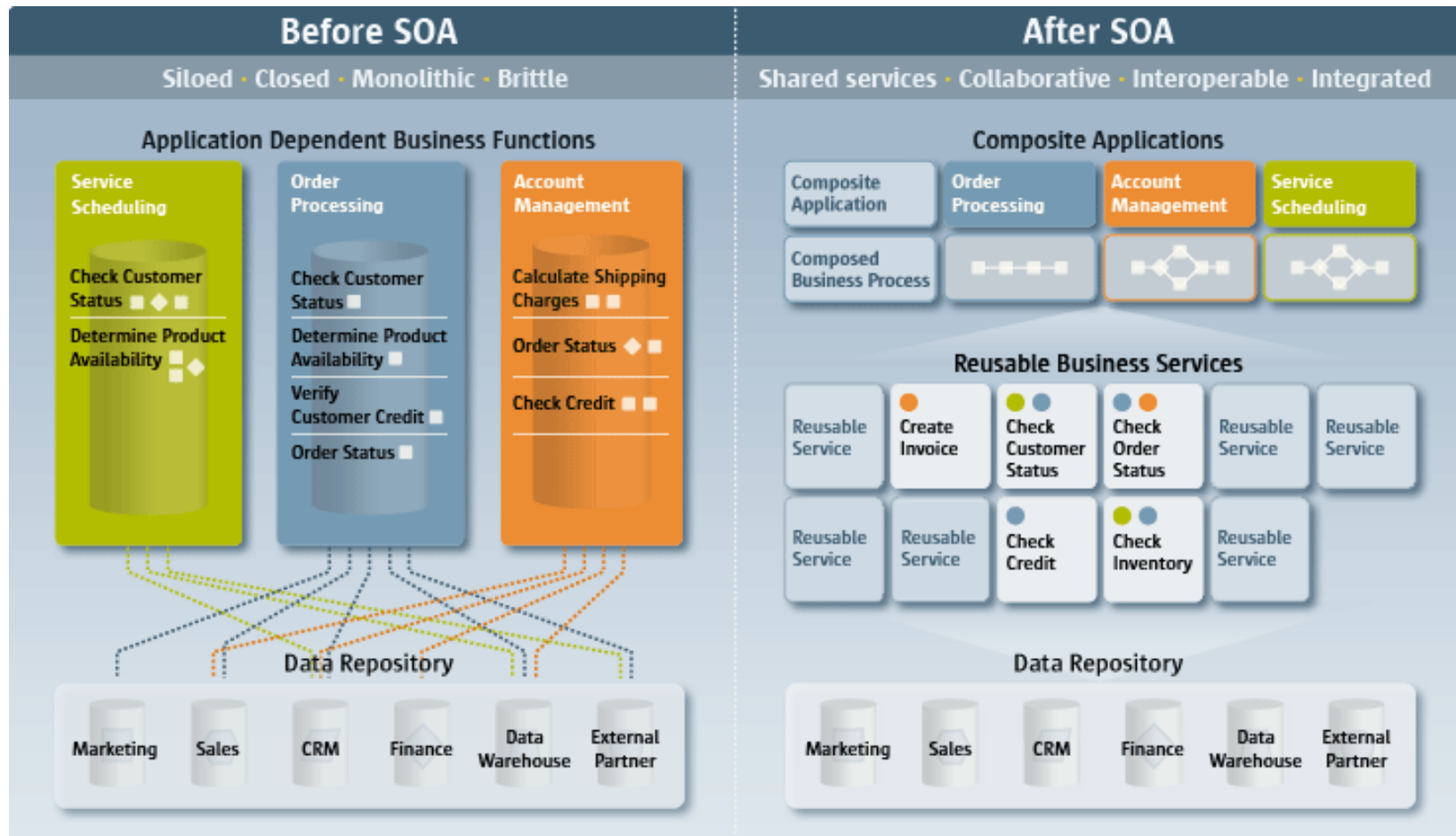
Services have explicit boundaries



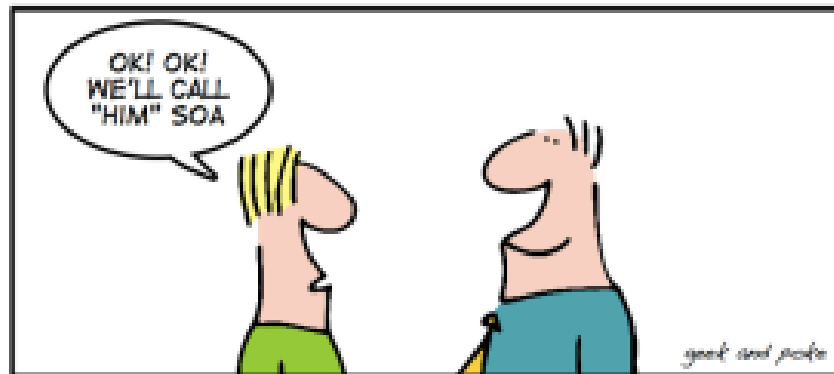
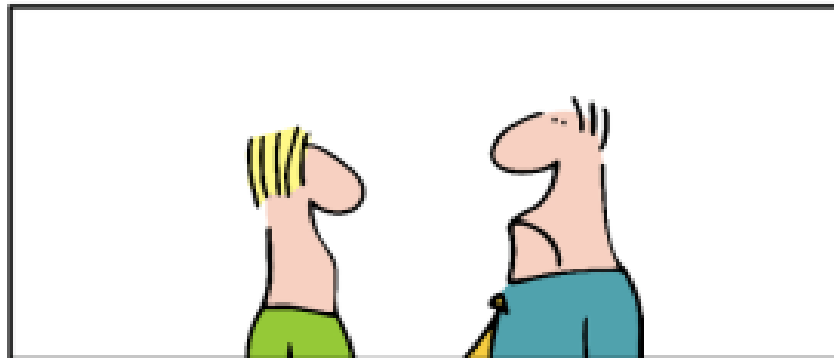
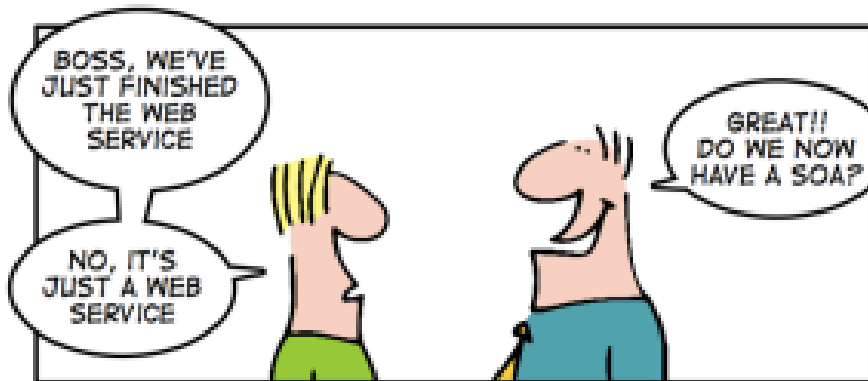
Services expose contract and policy



SOA !

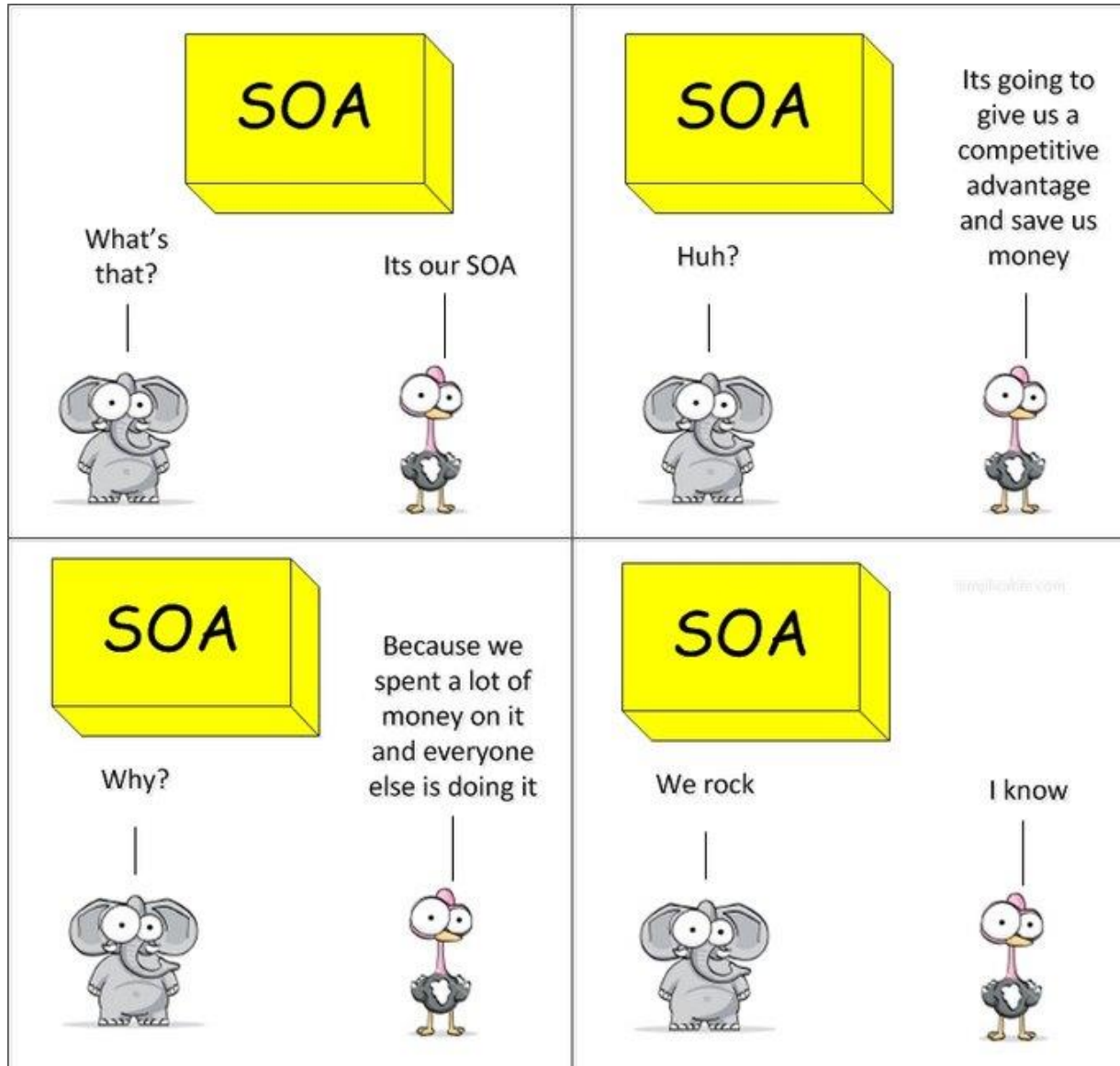


?



HOW TO GET A SOA

???



simplicable.com

What is REST?

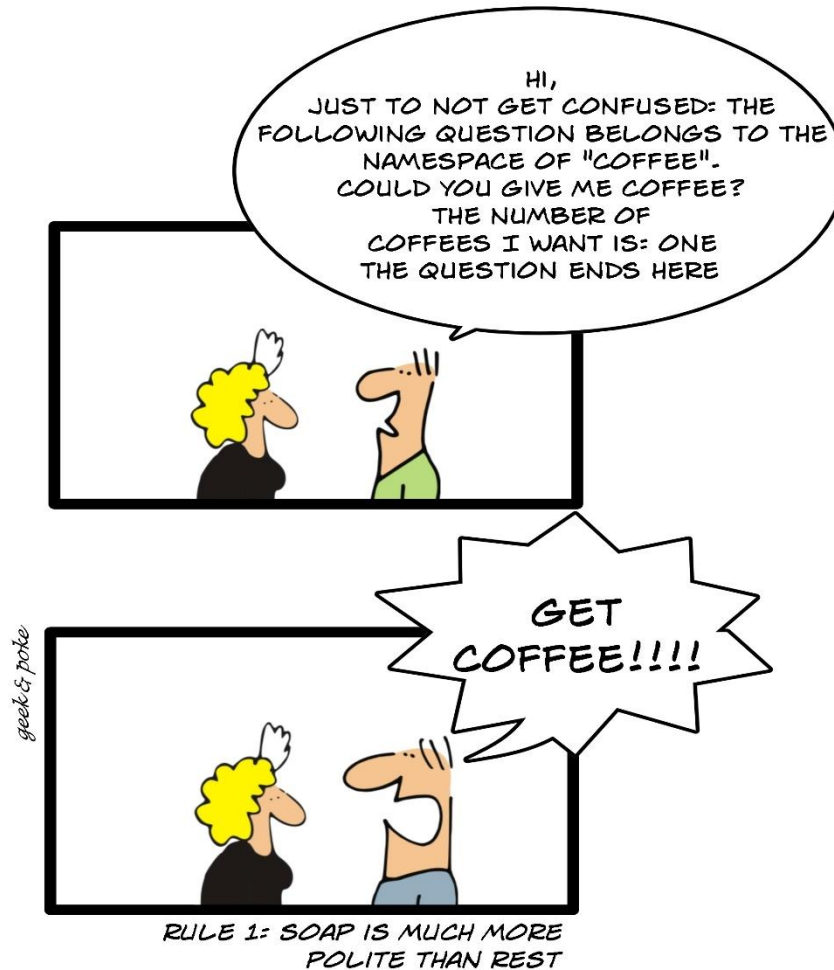
- REST stands for **Representational State Transfer** and is an architectural style. It was developed in parallel with HTTP/1.1. In short REST is an architectural style that exploits the existing web protocols and technologies. **REST provides an easier way of data access comparing with the SOAP protocol.** REST and SOAP relies on HTTP protocol. While considering interoperability of WCF with other platforms like Android, Iphone ... REST is considered to be a good solution.
- REST provides an alternative to SOAP and WSDL. REST on Web Services has evolved in big numbers for the last few years. Thomas Roy Fielding, the principal author of HTTP specification described REST as the key architectural principle of World Wide Web.

SOAP vs REST

- One of the key characteristics of a RESTful Web service is the explicit use of HTTP methods (GET, POST, PUT, DELETE) in a way that follows the protocol as defined by RFC 2616. But in the case of SOAP, POST is used to achieve all the cases.

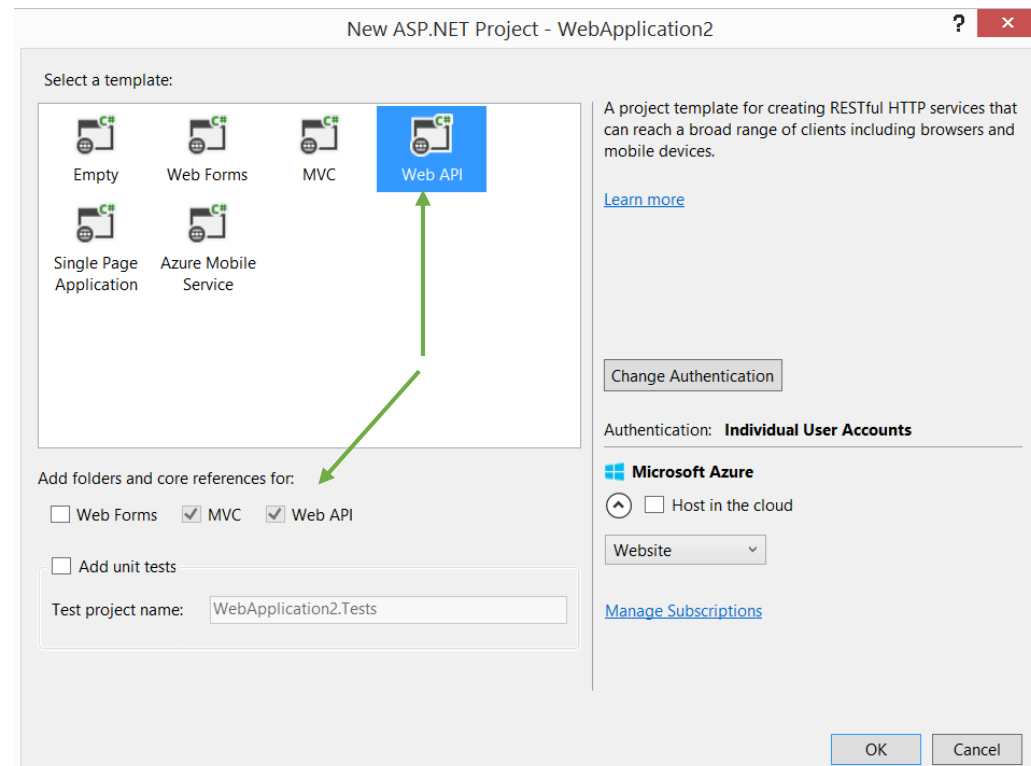
Method	Description	Example	SOAP (WS-*) Equivalent
GET	Retrieve an Item	GET /books/101	POST endpointInvoke GetOrder(101);
POST	Place an Item	POST /books <book> <id>102</id> <name>Book 2</name> <author>Author 2</author> <price>0</price> </book>	POST endpointInvoke InsertOrder(order);
PUT	Modify an Item	PUT /book/101	POST endpoint Invoke UpdateOrder(order);
DELETE	Delete an Item	DELETE /books/101	POST endpoint Invoke DeleteOrder(101);

SERVICE CALLING MADE EASY



ASP.NET Web API

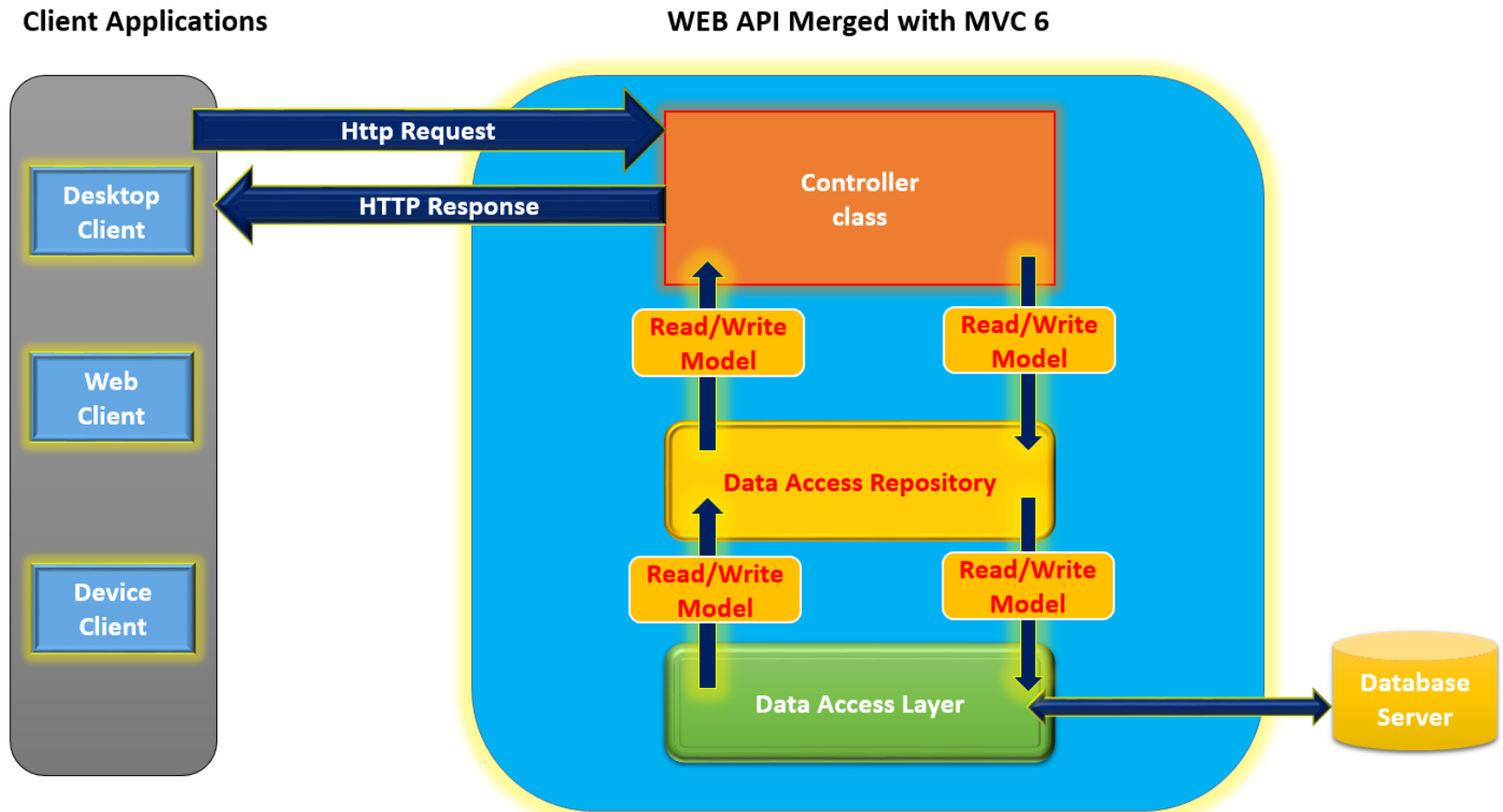
- Ein mögliches Backend für Web und Desktop Apps
- Erstellung von HTTP-basierten Services leicht möglich
- Ideal für RESTful APIs



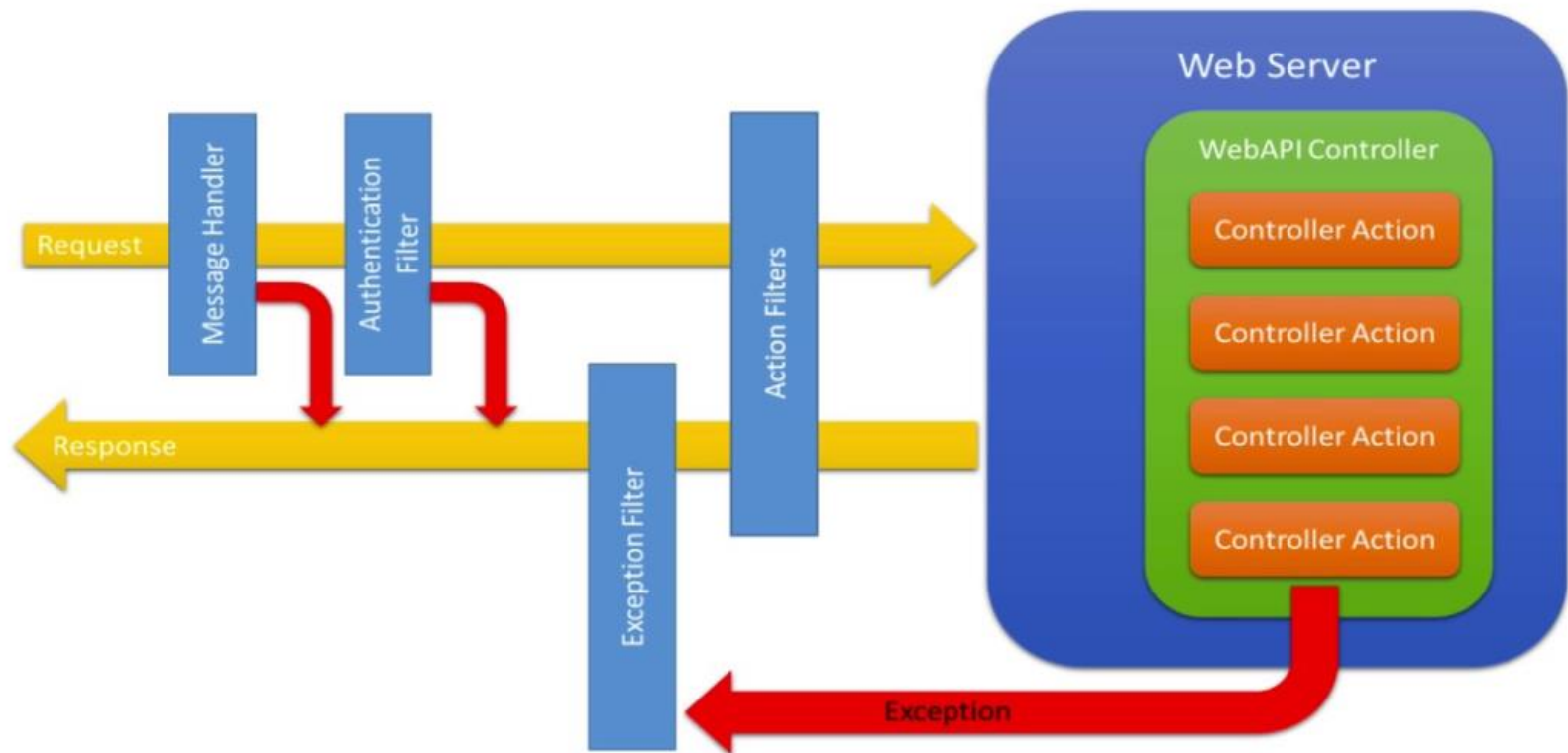
ASP.NET MVC Entwicklung

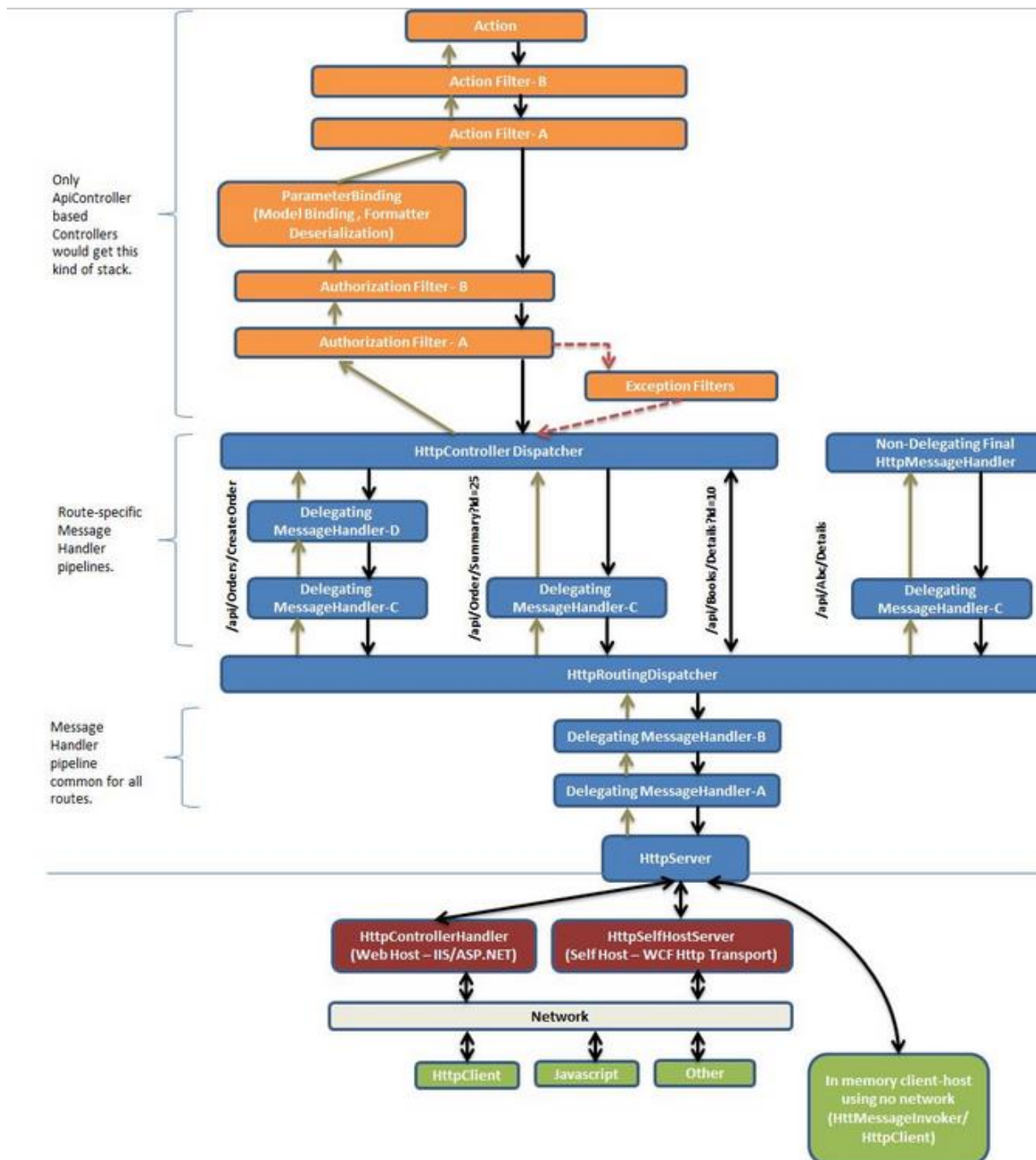
ASP.NET MVC3	ASP.NET MVC4	ASP.NET MVC5
▪ Templates for HTML 5 and CSS 3 ▪ Improved Model validation ▪ Razor View Engine ▪ Support for Multiple View Engines ▪ Controller improvements ▪ Unobtrusive <i>JavaScript</i> approach ▪ Improved Dependency Injection ▪ Partial page output caching http://www.webdevelopmenthelp.net	▪ ASP.NET Web API ▪ Adaptive rendering ▪ Look-n-Feel improvements ▪ Empty Project Template. ▪ Mobile Project Template ▪ Support for adding controller ▪ Task Support for Asyn Controllers ▪ Bundling and Minification ▪ Support for OAuth and OpenID ▪ Support for <i>Windows Azure SDK 1.6</i>	▪ ASP.NET Identity ▪ Authentication Filters ▪ Filter overrides ▪ Bootstrap ▪ Attribute Routing

Grundlagen MVC



Grundlagen MVC





REST

Representational State Transfer



Einführung REST

- **Die 4 Schichten des TCP/IP Referenzmodells**

Schicht	Name	Beschreibung
1	Netzzugangsschicht	Beschreibt Techniken zur Datenübertragung z.B. Verwendung der Netzwerkkarte – Protokolle: <i>Ethernet, FDDI, PPP, 802.11</i>
2	Internetschicht	Zuständig für die Weitervermittlung von Paketen – Protokoll: <i>IP (Internet Protocol)</i>
3	Transportschicht	Beschreibt wie Daten von einem Punkt zu einem anderen transportiert werden – Protokolle: TCP, UDP
4	Anwendungsschicht	Beschreibt die Bedeutung und das Format der transportierten Daten – Protokoll: HTTP

Einführung REST

„...the most efficient network request is one that doesn't use the network.“

- *Roy Fielding*

Einführung REST

- **Was ist REST ?**
 - Paradigma der Softwareentwicklung
 - Kein Protokoll wie HTTP oder SOAP



Einführung REST

- **Welche Eigenschaften muss ein RESTful Service haben ?**

- **Client Server Architektur**

- Eine einheitliche Schnittstelle trennt Client und Server.

- **Statelessness**

- Zustandslose Interaktion zwischen Client und Server.

- **Cacheability**

- Nachrichten vom Server (Responses) müssen sich selbst als cacheable oder nicht cacheable auszeichnen, damit der Client keine veralteten Daten zwischenspeichert.

Einführung REST

- **Welche Eigenschaften muss ein RESTful Service haben ?**

- **Layered System**

- Der Client hat keine Information darüber, ob er direkt mit dem finalen Server verbunden ist.

- **Uniform interface**

- Jeder Dienst, jede Methode oder jeder Status besitzt eine eindeutige Adresse.

- **Code on demand (*optional*)**

- Server kann die Funktionalität des Clients anpassen, indem ausführbarer Code mitgeliefert wird.



Einführung REST

- REST Operationen mit Hilfe von HTTP Verben

HTTP Methode	CRUD action	Beschreibung
GET	Read	Gibt einen einzelnen oder eine Sammlung von Einträgen zurück.
POST	Create	Erzeugen eines neuen Eintrags.
PUT	Update	Update (Aktualisierung) eines bestehenden Eintrags.
DELETE	Delete	Löschen eines Eintrags.
PATCH	Update	Teile eines Eintrags werden geändert / aktualisiert
Head	N/A	Liest die Message Header anhand der URI aus

Einführung REST

- Die wichtigsten HTTP Status Klassen
- HTTP-Status als Antwort möglich

HTTP Status	Beschreibung
1xx	Informativ
2xx	Erfolgreich (200 = OK, 201 = Created)
3xx	Umleitung
4xx	Client Error (400 = Bad Request, 404 = Not Found)
5xx	Server Error

Einführung REST

- **Schlüsselemente eines REST Service**

- **Uniform Resource Identifier (*URI*)**

- Lokalisiert eine spezifische Ressource über eine Zugriffsmethode (bspw. HTTP) in einem Netzwerk.

- **Links**

- Server stellt bearbeitbare Ressource mit Hilfe eines Hypermedia Links zur Verfügung. Konzept: *Hypermedia as the Engine of Application State (HATEOAS)*.

- **Uniform Interface**

- Service verwendet HTTP Verben, um auf adressierbare Ressourcen über eine URI zugreifen zu können.

Einführung REST

- **Schlüsselemente eines REST Service**

- **Stateless Communication**

- Zustandslose Kommunikation: Client muss Informationen im Request zur Verfügung stellen, damit Server eine entsprechende Response liefern kann.

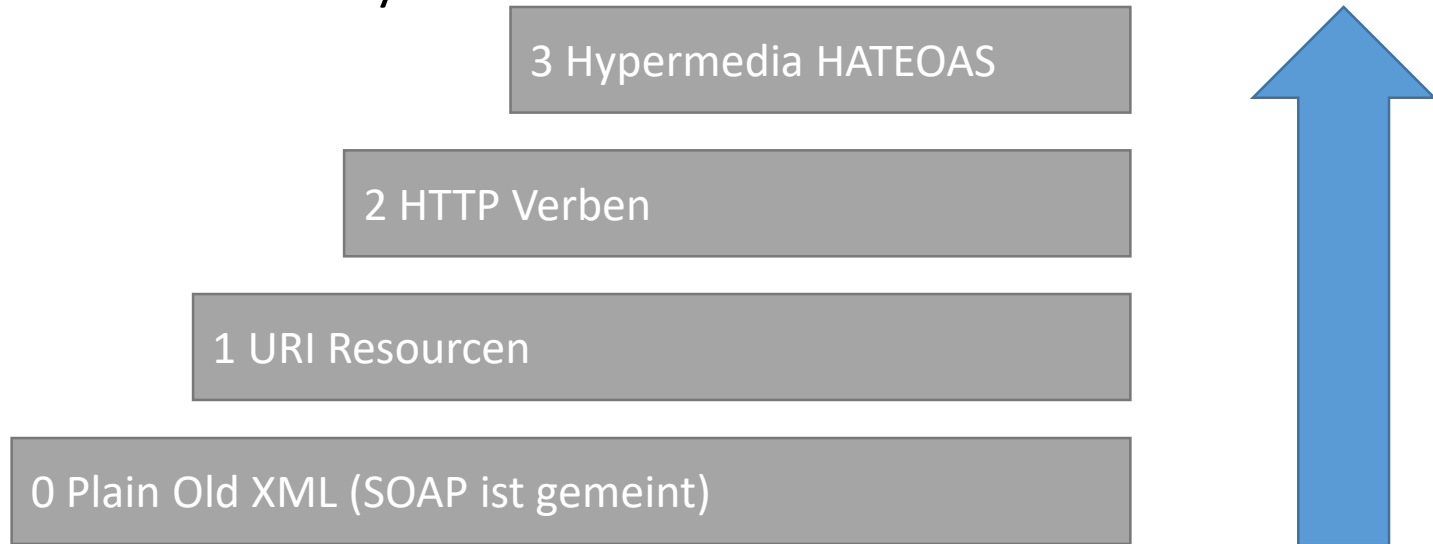
- **Mehrfache Repräsentationen einer Ressource**

- Jeder Service kann verschiedene „Repräsentationen“ einer Datenquelle liefern, wie beispielsweise JSON oder XML.



Einführung REST

- Richardson Maturity Model



Hypermedia as the Engine of Application State

Hypermedia as the Engine of Application State (HATEOAS)

- Hypermedia as the Engine of Application State (HATEOAS) is a constraint of the REST application architecture that distinguishes it from other network application architectures.
- With HATEOAS, a client interacts with a network application that application servers provide dynamically entirely through hypermedia. A REST client needs no prior knowledge about how to interact with an application or server beyond a generic understanding of hypermedia.
- By contrast, clients and servers in some service-oriented architectures (SOA) interact through a fixed interface shared through documentation or an interface description language (IDL).
- The way that the HATEOAS constraint decouples client and server enables the server functionality to evolve independently.

HATEOAS

- A REST client enters a REST application through a simple fixed URL. All future actions the client may take are discovered within resource representations returned from the server. In this way, RESTful interaction is driven by hypermedia, rather than out-of-band information.

```
GET /accounts/12345 HTTP/1.1
Host: bank.example.com
Accept: application/xml
```

```
HTTP/1.1 200 OK
Content-Type: application/xml
Content-Length: ...

<?xml version="1.0"?>
<account>
  <account_number>12345</account_number>
  <balance currency="usd">100.00</balance>
  <link rel="deposit" href="https://bank.example.com/accounts/12345/deposit" />
  <link rel="withdraw" href="https://bank.example.com/accounts/12345/withdraw" />
  <link rel="transfer" href="https://bank.example.com/accounts/12345/transfer" />
  <link rel="close" href="https://bank.example.com/accounts/12345/close" />
</account>
```


Einführung REST

- Original Dissertation über REST



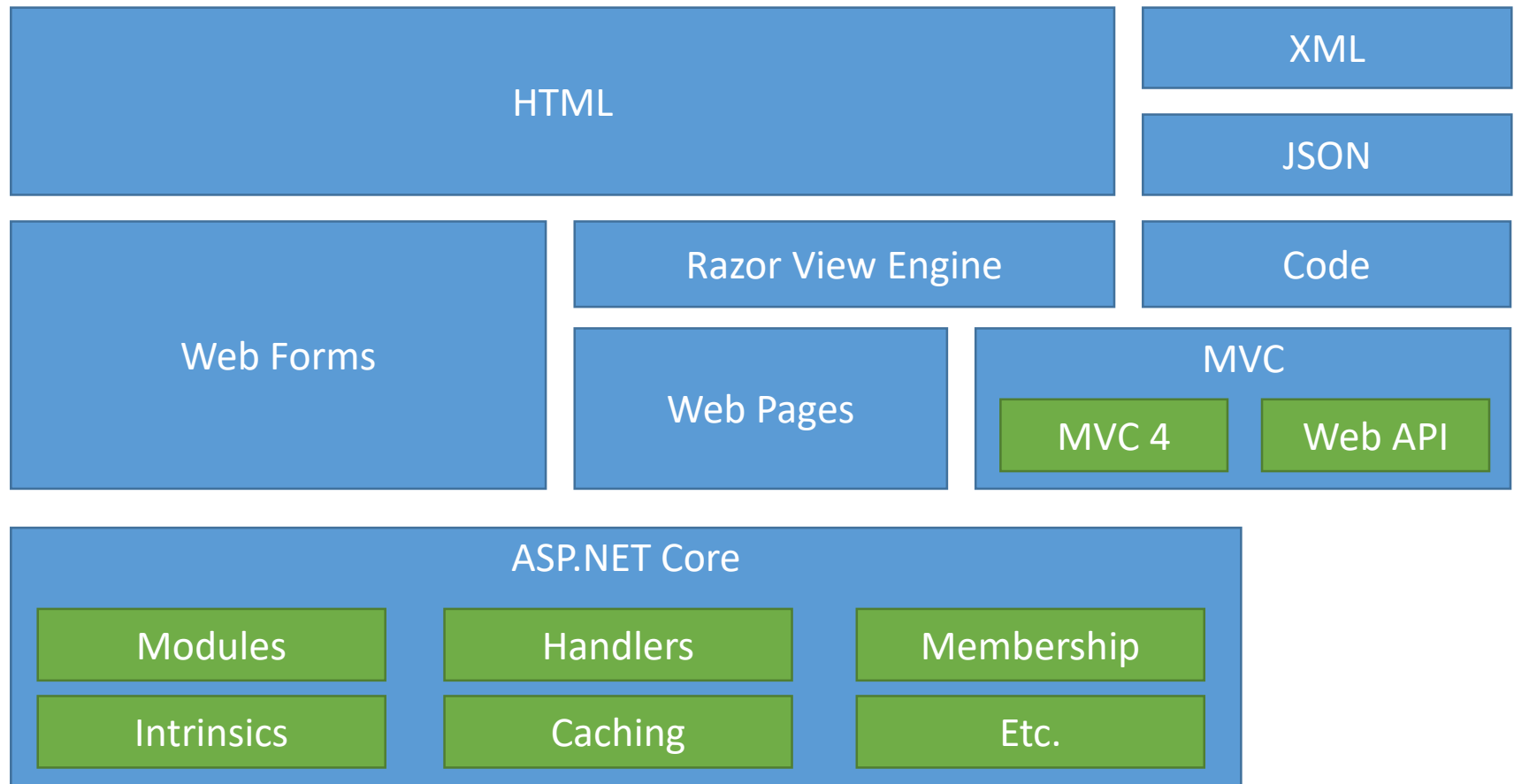
<http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

Web API 2.2

Das Microsoft Framework zur Erstellung von REST Services

Grundlagen ASP.NET Web API

- **Web API ist ein Teil von ASP.NET**



Grundlagen ASP.NET Web API

- **Was ist die ASP.NET Web API ?**
 - **Eine Bibliothek zur Erstellung von HTTP Services**
 - HTTP ermöglicht eine hohe Skalierbarkeit
 - Leichtgewicht aus dem Hause Microsoft
 - **Entwickelt, um REST Services leicht zu erstellen**
 - **Perfekt für heterogene Gerät/OS Szenarien**
 - Keine Probleme im Bezug auf Versionierung wie bei SOAP
 - **Mehrwert für das ASP.NET MVC 4 Modell**



Grundlagen ASP.NET Web API

- **Was kann die ASP.NET Web API?**
 - **Keine Bindung an Webanwendungen**
 - **Konvention über Konfiguration**
 - Verwendung von HTTP Methoden und HTTP Status Codes
 - **Content Negotiation**
 - „Inhaltsvereinbarung“ zwischen Client-Server
 - Die Web API gibt das geforderte Ergebnis serialisiert in der Form aus, die vom Client als akzeptierte Form angegeben wird (XML, JSON, ...).
 - Steuerung über den Request Header: Client sagt was er versteht
 - **Caching**
 - Unterstützt Standard HTTP Caching Mechanismus



Grundlagen ASP.NET Web API

- **Wo kann die ASP.NET Web API verwendet werden?**
 - Backend für Mobile Clients
 - Backend für Single Page Applikation
 - Backend für Desktop Anwendungen
 - Kann ASP.NET MVC und Web Forms ergänzen



Grundlagen ASP.NET und Web API

- **Web API VS. WCF**

- WCF ist ein Remote-Procedure Call und ruft Funktion auf entfernter Maschine auf
- Web API ist ein Rest Call und liefert immer das gleiche Ergebnis
- Nachteil WCF: Contract First Design und setzt ein Interface voraus
- Vorteil Web API: Ungebunden an UI, Dokument werden geholt
- Für HTTP Services ist Web API zu bevorzugen
- Bei TCP, Named Pipes, UDP, etc. scheidet Web API jedoch aus



Grundlagen ASP.NET und Web API

- **Wie kann Web API 2.2 installiert werden?**
 - Download NuGet Packages
 - ASP.NET Web API 2.1 ab Visual Studio 2013 Update 2
 - .NET 4.5+ erforderlich
 - Nützliche Links
 - Beispiele: <http://www.asp.net/web-api>
 - Sourcecode: <http://aspnetwebstack.codeplex.com/>

Web API und HTTP

- ‚Hello PPEDV‘ mit Web API 2.2
- Visual Studio → File → New Project
- Visual C# → Web → ASP.NET Web Application
- Vorlage: Web API, Change Authentication: No Authentication
- In der Datei Values Controller die GET Methode auf Hello PPEDV anpassen
- → CODE Snippets einbauen!!
- Fertige Methode pasten

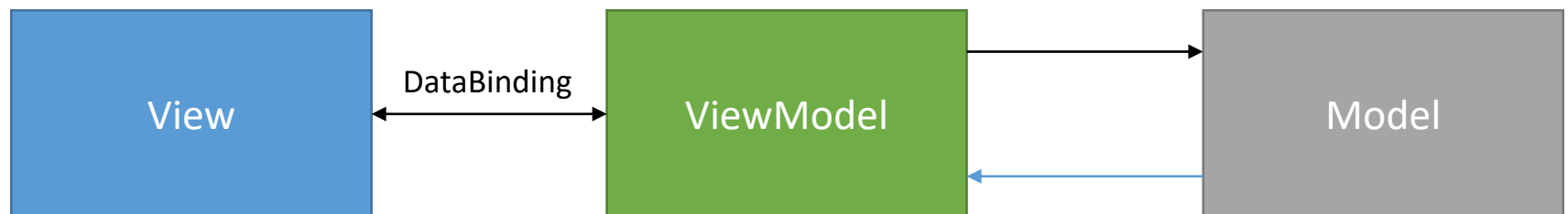


Demo

Web API und HTTP

- **Model – View – ViewModel (MVVM)**

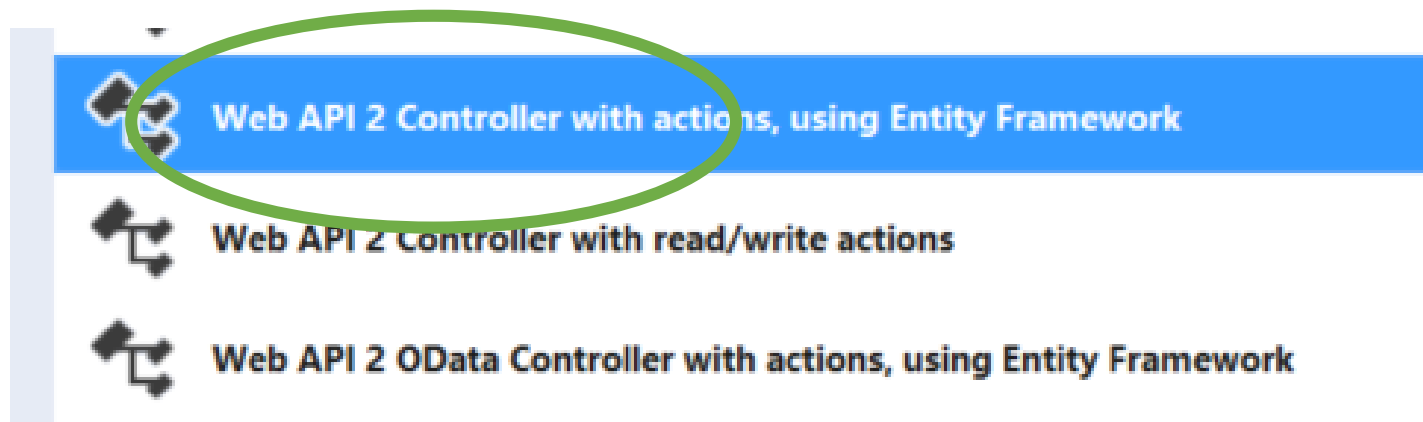
- View
 - „Darstellung“ - Inline-Code und Code-Behind
- ViewModel
 - „Vermittler“ Status und Operationen
- Model
 - Datenbasis



Web API und HTTP

- **API Controller**

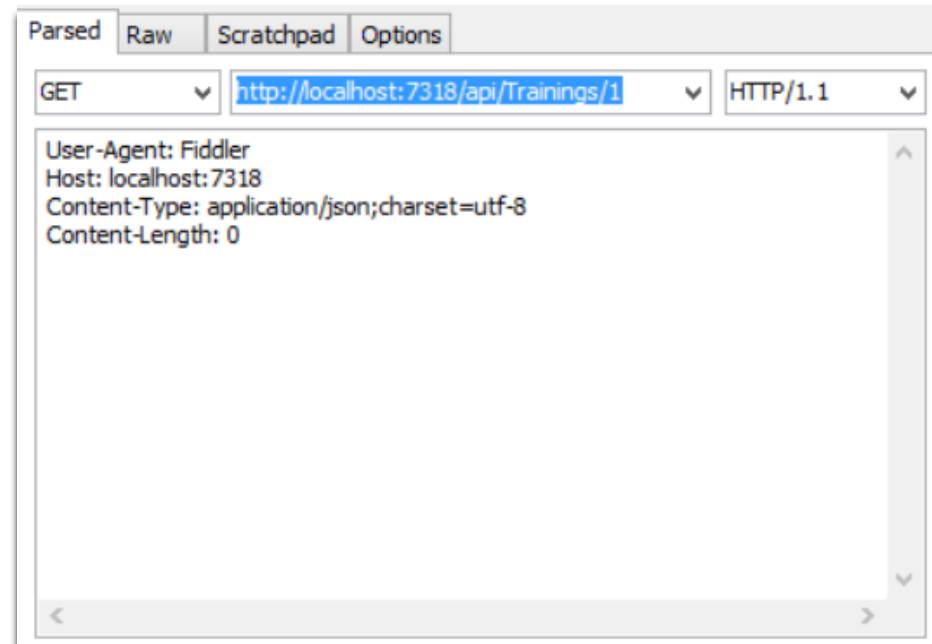
- Der Controller ist die Klasse, welche sämtliche Methoden enthält, die eine Anfrage (Request) entgegennehmen und beantworten können.
- Implementierung von: GET, POST, PUT, DELETE, ...



Web API und HTTP

- **Rest Tool - Fiddler**

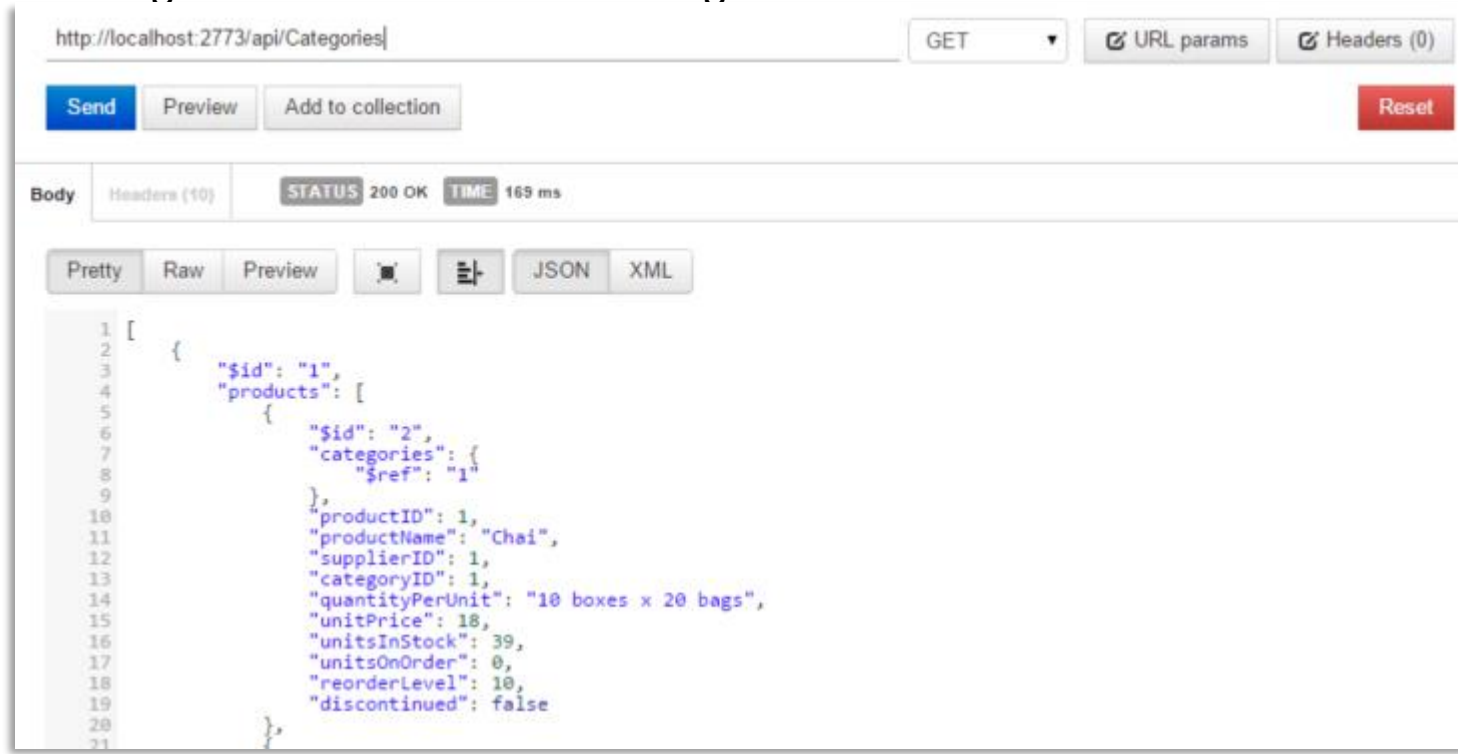
- <http://www.telerik.com/fiddler>
- Web Debugging
- Web Session Manipulation
- Performance Testing
- Security Testing
- HTTP/HTTPS Traffic Recording
- ...



Web API und HTTP

- **Rest Tool im Browser**

- Google Chrome mit Postman Plugin



0 Code Beispiel

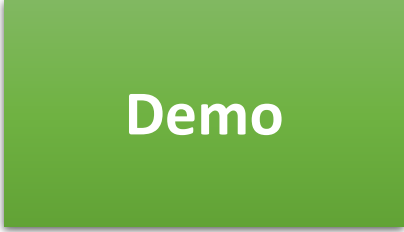
Web API mit Hilfe von Entity Framework (Version 6.1)



Web API und HTTP

- **ZeroCodeBeispiel**

- No Authentication
- CodeFirst from Database → Northwind
- (localdb)\v11.0
- CustomersSet
- Anpassen der Ausgabe



Demo

Web API und HTTP

The screenshot shows a web client interface with the following components:

- URL bar:** `http://localhost:1800/api/Customers/`
- Method:** `GET` (highlighted with a green circle)
- Buttons:** `Send`, `Preview`, `Add to collection`, `URL params`, `Headers (0)`, `Reset`
- Status:** `200 OK`, `TIME 176 ms`
- Response Body:** A JSON array of two customer objects.

```
1 [
2   {
3     "CustomerID": "ALFKI",
4     "CompanyName": "Alfreds Futterkiste",
5     "ContactName": "Maria Anders",
6     "ContactTitle": "Sales Representative",
7     "Address": "Obere Str. 57",
8     "City": "Berlin",
9     "Region": null,
10    "PostalCode": "12209",
11    "Country": "Germany",
12    "Phone": "030-0074321",
13    "Fax": "030-0076545"
14  },
15  {
16    "CustomerID": "ANATR",
17    "CompanyName": "Ana Trujillo Emparedados y helados",
18    "ContactName": "Ana Trujillo",
19    "ContactTitle": "Owner",
20    "Address": "Avda. de la Constitución 2222",
21    "City": "México D.F.",
22    "Region": null,
23    "PostalCode": "05021",
24    "Country": "Mexico",
25    "Phone": "(5) 555-4729",
26    "Fax": "(5) 555-3745"
27  }
28 ]
```


Web API und HTTP

The screenshot displays a REST client interface with the following components:

- URL Bar:** Contains the URL `http://localhost:1800/api/Customers/ALFKI`. The word **KEY** is written below the `ALFKI` segment, which is circled in green.
- Method:** A dropdown menu shows `GET`, which is also circled in green. The word **GET** is written below the dropdown.
- Buttons:** `URL params` and `Headers (0)` are on the right. `Send`, `Preview`, and `Add to collection` are below the URL bar. A red `Reset` button is on the far right.
- Response Section:**
 - Body:** The active tab.
 - Headers (10):** A tab for viewing headers.
 - STATUS:** `200 OK`
 - TIME:** `167 ms`
- Format Buttons:** `Pretty` (selected), `Raw`, `Preview`, `JSON`, and `XML`.
- Response Body:** A JSON object displayed in a code editor:

```
1 {  
2   "CustomerID": "ALFKI",  
3   "CompanyName": "Alfreds Futterkiste",  
4   "ContactName": "Maria Anders",  
5   "ContactTitle": "Sales Representative",  
6   "Address": "Obere Str. 57",  
7   "City": "Berlin",  
8   "Region": null,  
9   "PostalCode": "12209",  
10  "Country": "Germany",  
11  "Phone": "030-0074321",  
12  "Fax": "030-0076545",  
13 }
```

Web API und HTTP

- **Content Negotiation**

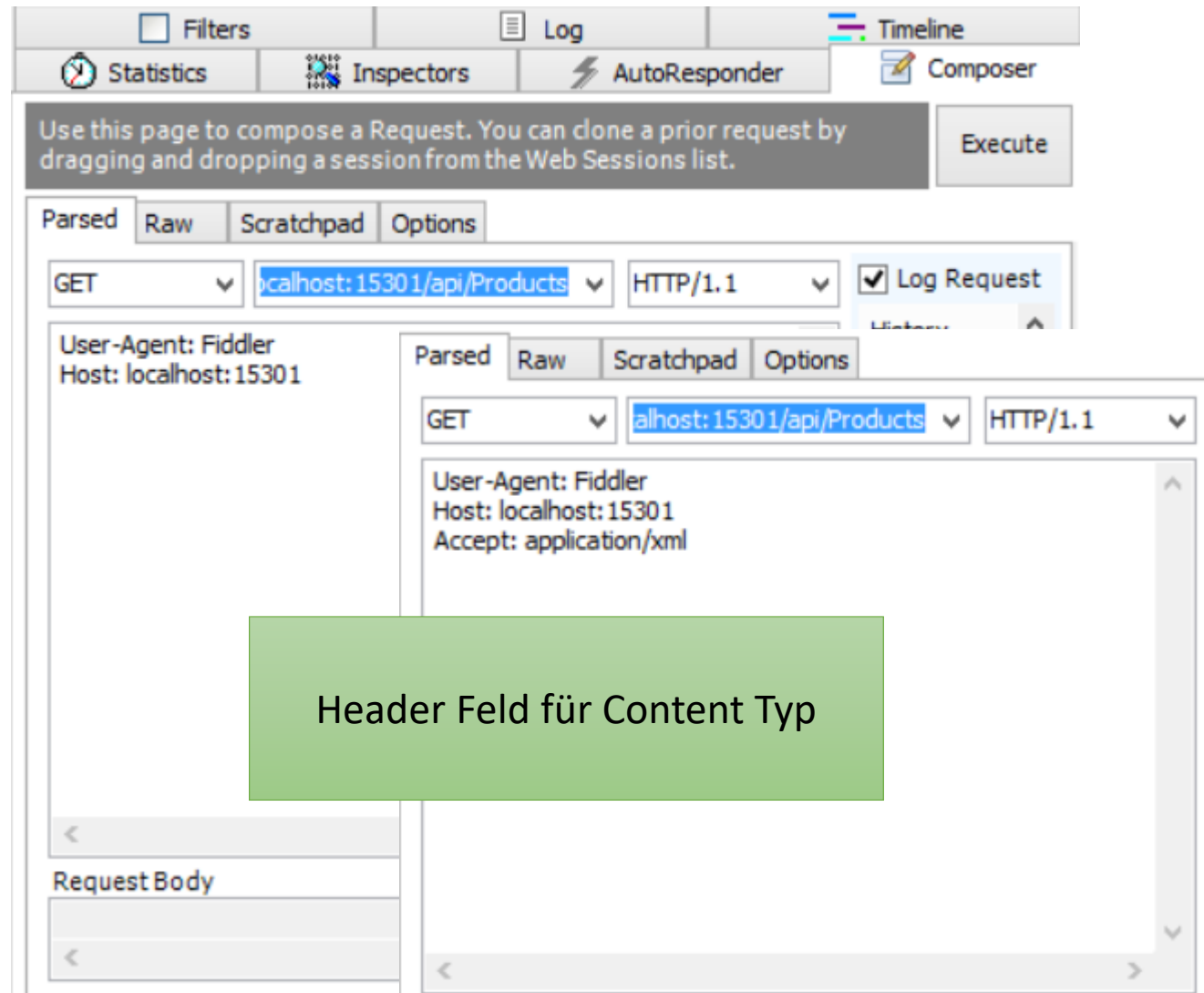
- „Inhaltsvereinbarung“ zwischen Client-Server
- XML oder JSON im **Accept** header
- Wird ein Format nicht gefunden: **406 Not Acceptable**
- XML basiert auf der ASP.NET Klasse **XmlMediaTypeFormatter**

- **Accept Header**

- Accept: application/xml
- Accept: application/json; charset=utf-8
- Für POST und PUT „Content-Type“ statt „Accept“ Attribut

Web API und HTTP

- Fiddler



Web API und HTTP

- Fiddler

The screenshot shows the Fiddler web proxy tool. The 'Inspectors' pane is active, displaying an XML response from a GET request to /api/Products. The XML structure is as follows:

```
ArrayOfProduct [ xmlns:i=http://www.w3.org/2001/XMLSchema-instance xmlns=http://s...  
  Product  
    Category  
      Id  
        1  
      Name  
        Getränke  
      CategoryId  
        1  
      Description  
        Stilles Wasser  
      Id  
        1  
      Name  
        Wasser  
  Product  
    Category  
      Id  
        1  
      Name  
        Cola  
      CategoryId  
        1  
      Description  
        Coca Cola  
      Id  
        2  
      Name  
        Cola
```

A green box with the text "JSON oder XML im Inspector" is overlaid on the right side of the XML view.

Web API und HTTP

- **POST Request / Daten schreiben**

The screenshot shows a REST client interface with the following elements:

- URL:** `http://localhost:1800/api/Customers/`
- Method:** **POST** (circled in green)
- Host:** `localhost:1800`
- Content-Type:** `application/json; charset=utf-8` (circled in green)
- Header:** A table with columns "Header" and "Value".
- Manage presets** button.
- Formatters:** `form-data`, `x-www-form-urlencoded`, `raw` (selected), and `Text` (dropdown).
- JSON Body:** A JSON object (circled in green) with the following fields:

```
1 {
2   "CustomerID": "NEWKI",
3   "CompanyName": "Neumann GmbH",
4   "ContactName": "Charlotte Musterfrau",
5   "ContactTitle": "Verkauf",
6   "Address": "Weg 7",
7   "City": "Burghausen",
8   "Region": null,
9   "PostalCode": "12345",
10  "Country": "Germany",
11  "Phone": "030-0074321",
12  "Fax": "030-0076545"
13 }
```
- Buttons:** `Send` (blue), `Preview`, `Add to collection`, and `Reset` (red).

Datensatz

Web API und HTTP

- PUT Request / Daten updaten

http://localhost:1800/api/Customers/NEWKI PUT URL params Headers (2)

Host localhost:1800 PUT

Content-Type application/json;charset=utf-8

Header	Value
--------	-------

Manage presets

form-data x-www-form-urlencoded raw Text

```
1 {
2   "CustomerID": "NEWKI",
3   "CompanyName": "Neumann GmbH",
4   "ContactName": "Manfred Mustermann",
5   "ContactTitle": "Verkauf",
6   "Address": "Weg 10",
7   "City": "München",
8   "Region": null,
9   "PostalCode": "12345",
10  "Country": "Germany",
11  "Phone": "030-0074321",
12  "Fax": "030-0076545"
13 }
```

Datensatz

Send Preview Add to collection Reset

Web API und HTTP

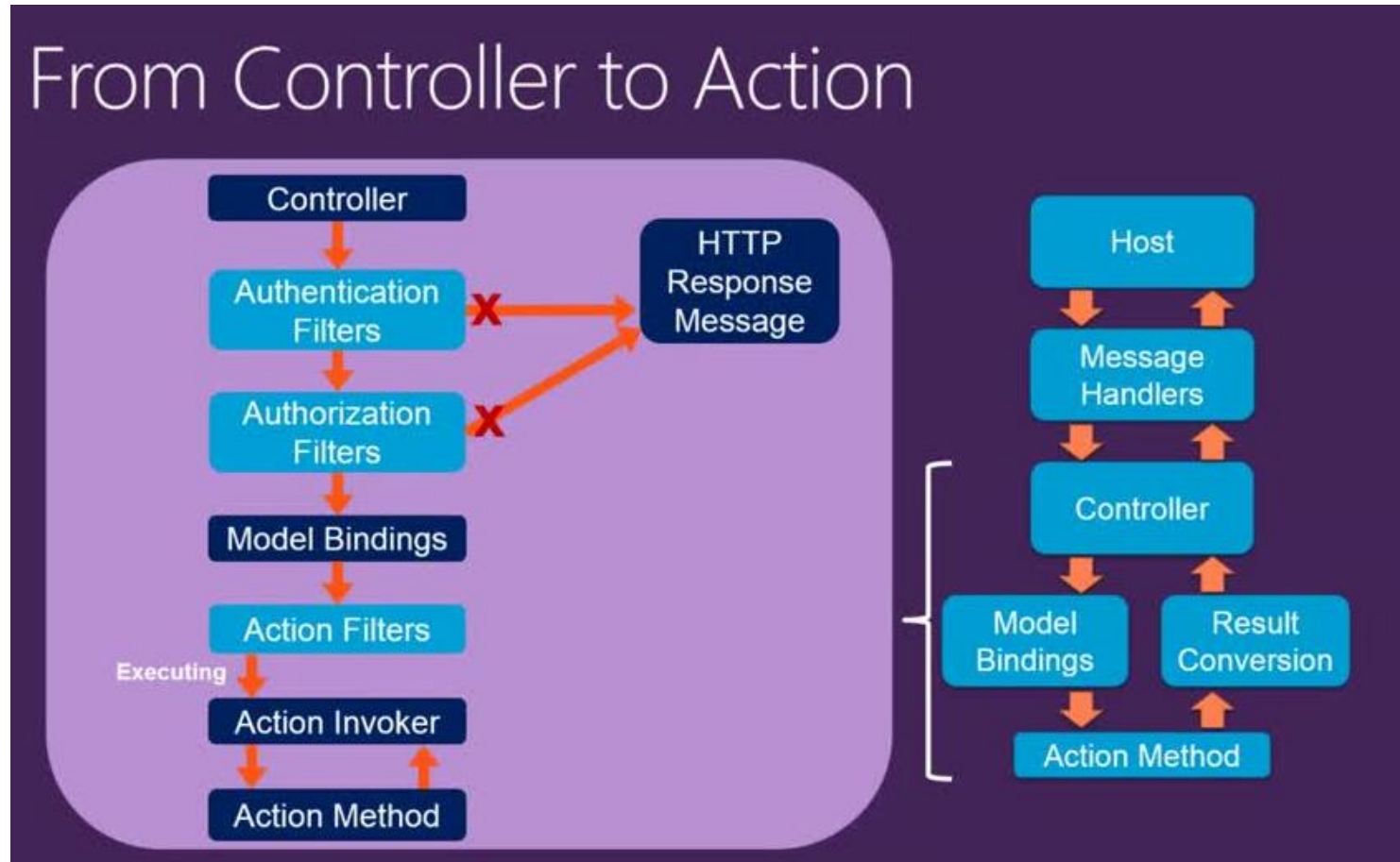
- **DELETE Request / Daten löschen**

The screenshot shows a REST client interface with the following elements:

- URL bar:** Contains the URL `http://localhost:1800/api/Customers/NEWKI` and a dropdown menu set to `DELETE`. To the right are buttons for `URL params` and `Headers (2)`.
- Host field:** Labeled **Host** with the value `localhost:1800`. A green circle highlights the word **DELETE** next to it.
- Content-Type field:** Labeled **Content-Type** with the value `application/json; charset=utf-8`.
- Header field:** Labeled **Header** with the value `Value`.
- Manage presets** button.
- Form tabs:** `form-data`, `x-www-form-urlencoded`, `raw`, and `Text` (selected).
- Request body:** A large text area with the number `1` and a cursor.
- Buttons:** `Send` (blue), `Preview` (grey), `Add to collection` (grey), and `Reset` (red).

Filter für WebAPI-Actions

From Controller to Action



Formatierung

Erweiterte Techniken



Web API Erweiterte Techniken

- **Formatting von Ressourcen**

- Formatter Klassen können :
- Daten lesen und schreiben
- Konvertieren von einem Format auf ein anderes

Members	Funktion
SupportedMediaType	Collection von Media Types, die vom Formatter unterstützt werden.
CanReadType	Boolean, gibt Auskunft darüber, ob der Formatter den Datentyp lesen kann.
CanWriteType	Boolean, gibt Auskunft darüber, ob der Formatter den Datentyp schreiben kann.
ReadFromStreamAsync	Wird beim lesen des Input Streams aufgerufen. Gibt die Ressource zurück.
WriteToStreamAsyng	Aufruf beim Schreiben der Ressource in den Output Stream.

Web API und HTTP

- „Out-of-the-box“ Media Type Formatters in ASP.NET

Media Formatter	Funktion
JsonMediaTypeFormatter	Formatiert die geforderte Ressource im JSON Format
XmlMediaTypeFormatter	Formatiert die geforderte Ressource im XML Format
FormUrlEncodedMediaTypeFormatter	Formular Requests: application/x-www-form-urlencoded
JQueryMvcFormUrlEncodedFormatter	Erbt von
FormUrlEncodedMediaTypeFormatter	Und fügt Unterstützung zu JQuery hinzu

- Das Erstellen von eigenen Formattern ist in ASP.NET Web API möglich

Web API und HTTP

- Verzeichnis App_Start → WebApiConfig.cs

```
// Web API configuration and services
config.Formatters.JsonFormatter
    .SerializerSettings.ContractResolver =
        new CamelCasePropertyNamesContractResolver();
```



HTTP Statuscodes

Erweiterte Techniken

Web API Erweiterte Techniken

Code	Nachricht	Bedeutung
200	OK	Erfolg, das Ergebnis der Anfrage wird in der Antwort übertragen.
201	Created	Erfolgreich bearbeitet, die Ressource wurde vor dem Senden der Antwort erstellt.
202	Accepted	Anfrage akzeptiert, wird aber erst später ausgeführt. Kein Gelingen garantiert
301	Moved Permanently	Redirect, die angeforderte Ressource steht unter einer anderen Adresse bereit. Die alte Adresse ist nicht länger gültig.
304	Not Modified	Mit Web Api Caching
400	Bad Request	Fehlerhafte Anfrage
401	Unauthorized	Keine gültige Authentifizierung, Anfrage nicht möglich.
402	Payment Required	Status für zukünftige HTTP-Protokolle reserviert – „ <i>Bezahlung benötigt</i> “. Wird oft verwendet um Fehler im Validierungsprozess aufzudecken.
403	Forbidden	Client nicht berechtigt, Anfrage wurde nicht durchgeführt.
404	Not Found	Die angeforderte Ressource wurde nicht gefunden. Anfrage ohne Grund abgewiesen.
409	Conflict	Anfrage wurde unter falschen Annahmen gestellt.
500	Internal Server Error	Unerwarteter Serverfehler.

Web API und HTTP

- **Rückgabe HTTP Status Codes**
- *HTTP Action Result* seit Version 2 als Rückgabetyt möglich!
- Anpassen des Controllers
 - Wenn GET Ressource nicht gefunden wurde, HTTP **404 Not Found** als Rückgabecode
 - POST, PUT, DELETE mit Rückgabetyt *IHttpActionResult* anpassen
- Prüfung des Formats mit HTTP Debugger



Demo

Async und Await

Erweiterte Techniken



Web API Erweiterte Techniken

- **Asynchrone (Server) Entwicklung**

- „Reaktionsfähigkeit“ von Anwendungen verbessern
- „Nicht blockierendes“ Programmieren
- `async` und `await` seit C# 5.0
- `async` deklariert Methode, die in eigenen Thread ausgelagert wird und als Rückgabebetyp *Task* oder *void* verwenden muss
- `await` wird beim Beginn des asynchronen Codeblocks verwendet und gibt kontrolliert Thread an dieser Stelle
- Ausführung von asynchronem Code im Controller

Don't Block on Async Code

Consider the example below. A button click will initiate a REST call and display the results in a text box (this sample is for Windows Forms, but the same principles apply to *any* UI application).

```
// My "library" method.
public static async Task<JObject> GetJsonAsync(Uri uri)
{
    using (var client = new HttpClient())
    {
        var jsonString = await client.GetStringAsync(uri);
        return JObject.Parse(jsonString);
    }
}

// My "top-level" method.
public void Button1_Click(...)
{
    var jsonTask = GetJsonAsync(...);
    textBox1.Text = jsonTask.Result;
}
```

The “GetJson” helper method takes care of making the actual REST call and parsing it as JSON. The button click handler waits for the helper method to complete and then displays its results.

This code will deadlock.

Don't Block on Async Code

ASP.NET Example

This example is very similar; we have a library method that performs a REST call, only this time it's used in an ASP.NET context (Web API in this case, but the same principles apply to *any* ASP.NET application):

```
// My "library" method.
public static async Task<JObject> GetJsonAsync(Uri uri)
{
    using (var client = new HttpClient())
    {
        var jsonString = await client.GetStringAsync(uri);
        return JObject.Parse(jsonString);
    }
}

// My "top-level" method.
public class MyController : ApiController
{
    public string Get()
    {
        var jsonTask = GetJsonAsync(...);
        return jsonTask.Result.ToString();
    }
}
```

This code will also deadlock. For the same reason.

What Causes the Deadlock ?

- After you await a Task, when the method continues it will continue in a context. In the first case, this context is a UI context (which applies to any UI except Console applications). In the second case, this context is an ASP.NET request context.
- One other important point: an ASP.NET request context is not tied to a specific thread (like the UI context is), but it does only allow one thread in at a time.



What Causes the Deadlock ?

So this is what happens, starting with the top-level method (Button1_Click for UI / MyController.Get for ASP.NET):

1. The top-level method calls GetJsonAsync (within the UI/ASP.NET context).
2. GetJsonAsync starts the REST request by calling HttpClient.GetStringAsync (still within the context).
3. GetStringAsync returns an uncompleted Task, indicating the REST request is not complete.
4. GetJsonAsync awaits the Task returned by GetStringAsync. The context is captured and will be used to continue running the GetJsonAsync method later. GetJsonAsync returns an uncompleted Task, indicating that the GetJsonAsync method is not complete.
5. The top-level method synchronously blocks on the Task returned by GetJsonAsync. This blocks the context thread.
6. ... Eventually, the REST request will complete. This completes the Task that was returned by GetStringAsync.
7. The continuation for GetJsonAsync is now ready to run, and it waits for the context to be available so it can execute in the context.
8. **Deadlock.** The top-level method is blocking the context thread, waiting for GetJsonAsync to complete, and GetJsonAsync is waiting for the context to be free so it can complete.

For the UI example, the “context” is the UI context; for the ASP.NET example, the “context” is the ASP.NET request context. This type of deadlock can be caused for either “context”.

Web API Erweiterte Techniken

- **Typische Anwendungsbereiche für async. Programmierung**

Anwendungsbereich	Unterstützende APIs
Webzugriff	HttpClient, SyndicationClient
Arbeiten mit Dateien	StorageFile, StreamWriter, StreamReader, XmlReader
Arbeiten mit Bildern	MediaCapture, BitmapEncoder, BitmapDecoder
WCF-Programmierung	Synchrone und asynchrone Vorgänge

Web API Erweiterte Techniken

- Umbau des Controllers auf Asynchrone Methoden
- Prüfung des Formats mittels HTTP Debugger



Demo

Caching

Erweiterte Techniken



Web API Erweiterte Techniken

- **HTTP-Caching**

- Performance optimieren!
- Ein Client kann Ressourcen „cachen“ und dadurch die Zugriffszeiten verbessern.
- Der Server muss über die „Cacheability“ einer Ressource Auskunft geben. → Kann eine Ressource gecached werden?
- Ein gutes Cacheing-Plugin von Brad Wilson
 - <https://gist.github.com/bradwilson/8586562>

Web API Erweiterte Techniken

- Einbau des Caching Mechanismus z.B. mit dem Web API Plugin von Brad Wilson
- Anpassen des Controllers
- Prüfung mit Browser – HTTP Debugger ignoriert Caching



Demo

Web API Erweiterte Techniken

```
// GET: api/Customers
public IHttpActionResult GetCustomers()
{
    return
        Ok(db.Customers)
        .Cached(Cacheability.Public, maxAge:
            TimeSpan.FromMinutes(15));
}
```



Binding

Erweiterte Techniken

Web API Erweiterte Techniken

- **Objektteile zurückliefern**

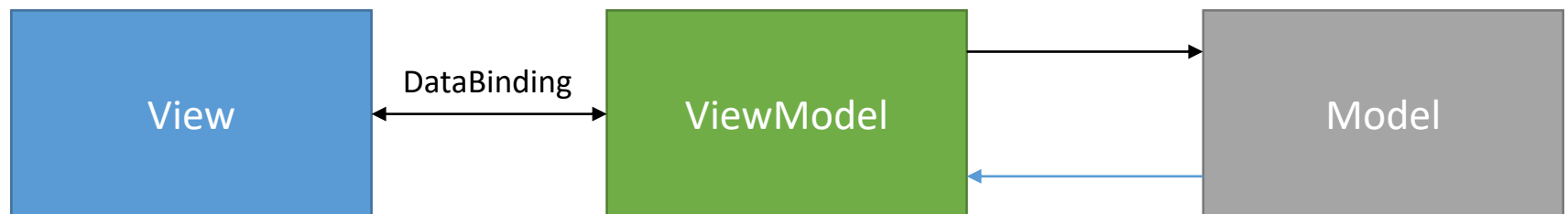
- Sinnvoll wenn weniger Daten ausgeliefert werden sollen
- Möglich über **Lambda Expression** = anonymes Objekt
- Oder über **View-Model** = „richtiges“ Objekt



Web API Erweiterte Techniken

- **Model – View – ViewModel (MVVM)**

- View
 - „Darstellung“ - Inline-Code und Code-Behind
- ViewModel
 - „Vermittler“ Status und Operationen
- Model
 - Datenbasis



Web API Erweiterte Techniken

- **Lambda Expression**

```
public async Task<IHttpActionResult> Get()
{
    using (var db = new ProductsContext ())
    {
        return Ok(await db.Products.Include(t => t.Category)
            .Select(t => new { Title = t.Name, Number = t.Id })
            .ToListAsync()).
        Cached(cacheability: Cacheability.Public,
            maxAge: TimeSpan.FromSeconds(30));
    }
}
```



Web API Erweiterte Techniken

```
// GET api/Products
public async Task<IHttpActionResult> Get()
{
    using (var db = new ProductsContext ())
    {
        return Ok(await db.Products.Include(t => t.Category)
            // Übergabe ViewModel auf Basis der akt. Entität
            .Select(t => new ProductViewModel
            { Title = t.Name, Number = t.Id })
            .ToListAsync()).
            Cached(cacheability: Cacheability.Public,
            maxAge: TimeSpan.FromSeconds(30));
    }
}
```



Web API Erweiterte Techniken

```
// GET api/Products/5
public async Task<IHttpActionResult> Get(int id)
{
    using (var db = new ProductsContext())
    {
        var result = await db.Products.Include(t => t.Category)
            .Select(t => new ProductViewModel
            { Title = t.Name, Number = t.Id })
            .SingleOrDefaultAsync(t => t.Number == id);

        if (result == null)
        {
            return NotFound();
        }

        return Ok(result);
    }
}
```



Web API Erweiterte Techniken

- **Binding**

- Aufruf einer Controller Methode mit Übergabeparametern durch die Web API
- HttpResponseMessage Put(int id, Product item) { ... }
- Id = primitive Type
- Product = Complex Type

- Complex Type aus URI: **[FromUri]**
- Complex Type aus Body: **[FromBody]**



Routing

Erweiterte Techniken

Binding, Validation, Routing

- **Routing**

- Routing erfolgt in **Routing Tabellen**
- oder auch über **Attribute** für „temporäre“ Routs
- Zuweisen von Controller und Methoden zur passenden Anfrage

- **Routing in 3 Schritten**

- Vergleichen von URI und Route Template
- Auswahl eines Controllers
- Auswahl einer Action



Binding, Validation, Routing

- **Routing Tabelle**

- App_Start → WebApiConfig.cs

```
config.Routes.MapHttpRoute(  
    name: "ProductCategoryRoute",  
    routeTemplate: "api/product/{id}/products",  
    defaults: new { controller = "Products",  
                    action = "GetByCategory" },  
    constraints: new { id = @"\d+" }  
);
```

Binding, Validation, Routing

- **Attribute**

- Aufruf von `config.MapHttpAttributeRoutes()`:

```
// Selektion nach Kategorie Name, :alpha -> string!  
// api/category/Datenzugriff/products  
[ResponseType(typeof(Models.Product))]  
[Route("api/category/{name:alpha}/products")]  
public async Task<IHttpActionResult> GetCategory(string name)  
{  
    using (var db = new ProductsContext())  
    {  
        return Ok(await db.Products.Where(t => t.Category.Name == name)  
            .Include(t => t.Category)  
            .ToListAsync());  
    }  
}
```

Validation

Erweiterte Techniken



Binding, Validation, Routing

- **Validierung von eingehenden Daten**

- Mittels Attributen im Model z.B.:
 - [Required]
 - [MinLength(10)]
 - [MaxLength(255)]
 - [Range(1, int.MaxValue)]
 - ...
- Datenbank Updaten, um Exceptiones zu vermeiden!
- PMC:
Add-Migration ConstraintsErstellt,
Update-Database



Binding, Validation, Routing

- Controller, POST Methode anpassen
- Z.B.: HTTP Status Code 400 (Bad Request) zurück geben

```
// Validierung
if (!ModelState.IsValid)
{
    return BadRequest(ModelState);
}
```

Security

Erweiterte Techniken



Security

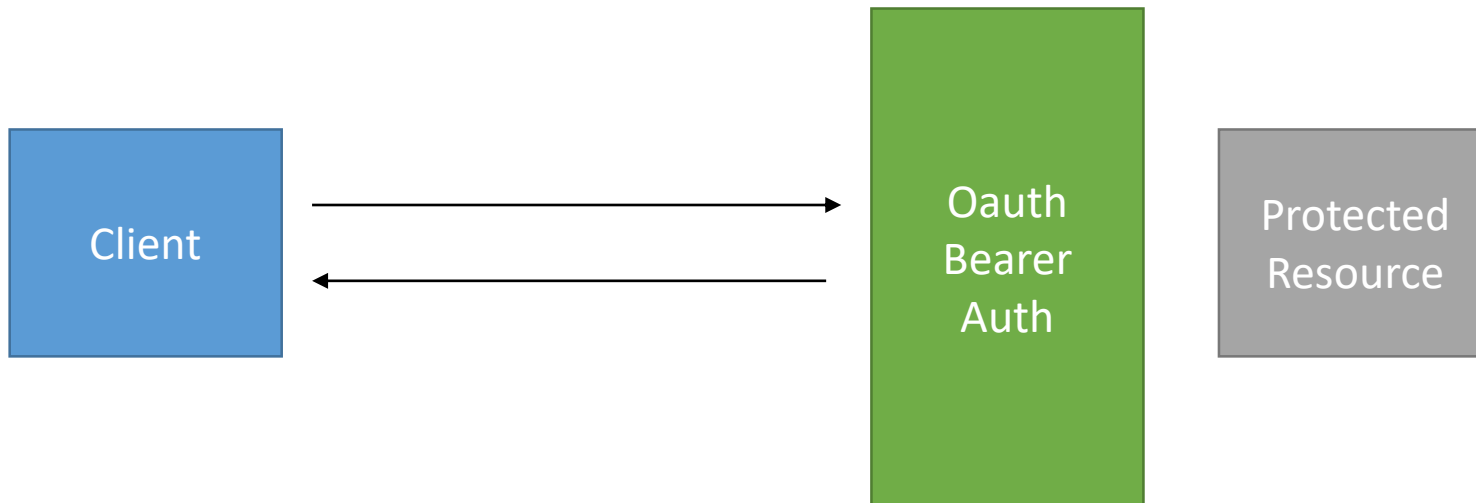
- **Unauthorisierten Zugriff auf die Web API unterbinden**
 - Verwendung von Authorization Filtern (z.B. [Authorize])
 - Implementierung der Authorization Logik
 - AuthorizationFilterAttribute oder Interface IAuthorizationFilter
 - Zuweisung: Global, am Controller oder auf der Action



Security

- **Oauth 2.0**

- Requests mit Oauth 2.0 Bearer Token absichern
- Bearer Token in Standard Authorization Header einbinden
- Middleware validiert und konvertiert „Token“ in „Claims“
- Voraussetzung: SSL Verbindung



Security

- **Tokens von Trusted Authorization Servern beziehen**
 - **Validierungsserver selbst hosten**
 - Eingebaute MS OWIN Komponenten zur Authorisierung
 - Verwenden des ThinkTecture AuthorizationServer
 - **Bestehende Lösungen Verwenden**
 - Windows Azure Active Directory
 - Active Directory Federation Services in Windows Server 2012 R2
 - Windows Azure Mobile Services



Security

- **Authentifizierung und Autorisierung**

- Attribut **[Authorize]**
- Authentifizierung mit OWIN (Middleware)
- Authentifizierung z.B. für POST, PUT, DELETE gut geeignet
- Authentifizierung über Nuget Pakete:
 - Microsoft.AspNet.Identity.Owin
 - Microsoft.AspNet.Identity.Entity
 - Microsoft.owin.host.systemweb

Security

- **CORS (Cross-Origin Resource Sharing)**

- Keine Domänen-übergreifende AJAX Datenübertragung erlaubt
- Cross-Origin Ressource Sharing aktivieren!
- Aktivieren: NuGet-Paket und Sourcecode:
 - Suche cors: Microsoft ASP.NET Web API 2.1 Cross-Origin Support
 - App_Start → WebApiConfig.cs → `config.EnableCors();`
 - Attribut über Klasse oder Methode

```
[EnableCors(origins: "*", headers: "*", methods: "*")]
```

OData

Open Data Protocol, Version 4
„SQL Light für das WEB“

OData

- **Aktuelle Problematik BIG DATA:**

- Eine zunehmende Anzahl an Informationen muss von einer steigenden Anzahl an unterschiedlichen Datenquellen auf den verschiedensten Geräten ganzheitlich bezogen werden können.

- **Einheitliches Web Protokoll - die Lösung**

- Ein Web Protokoll zum Abfragen, bearbeiten und hinzufügen von Daten auf Basis von Web Technologien, wie HTTP, XML und JSON

OData

- **Das Web Protokoll OData – Open Data Protokoll**

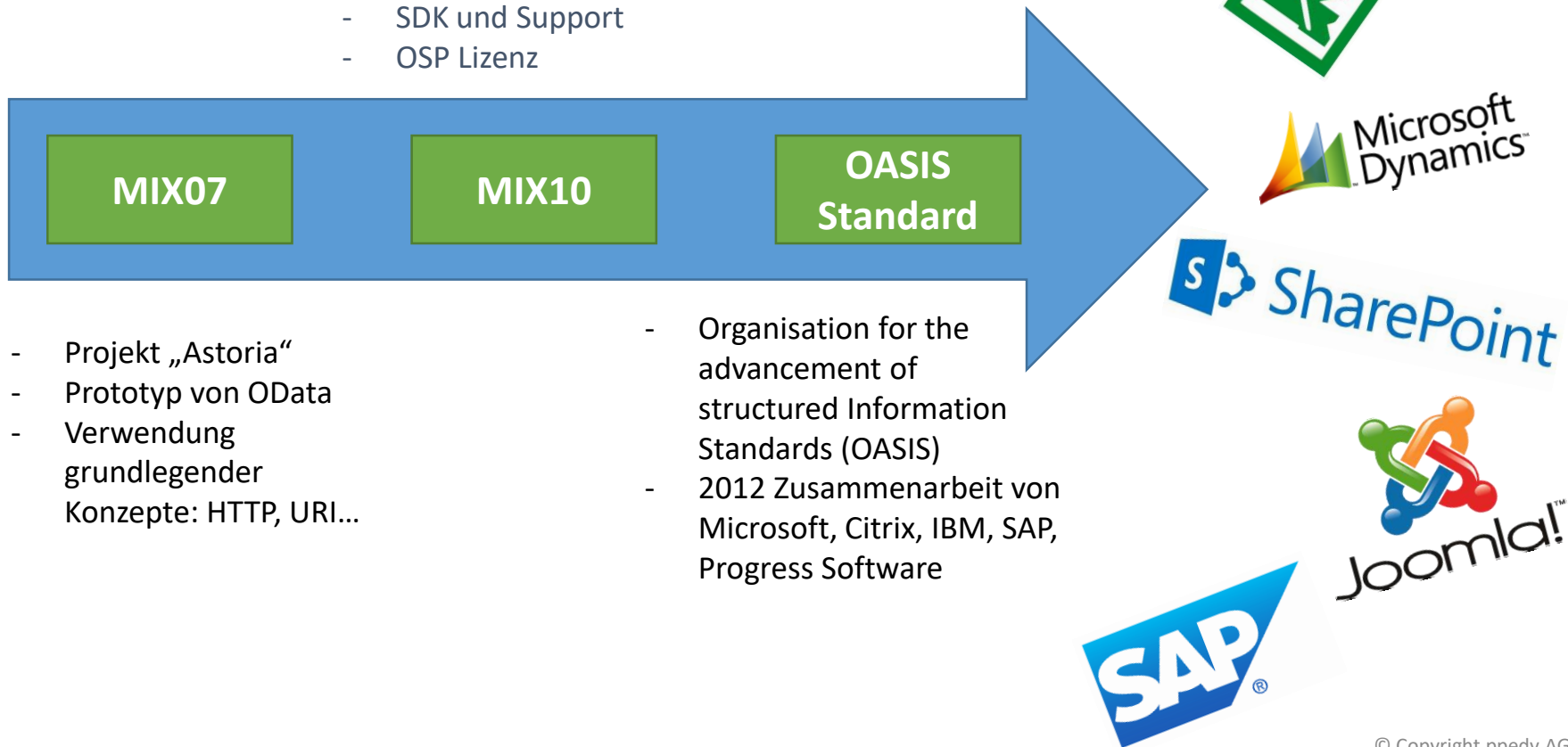
- REST basiertes, offenes Web Protokoll von Microsoft
- Ziel: Ressourcen über das HTTP Protokoll zur Verfügung stellen
- Basiert auf bekannten Standards: AtomPub, XML, JSON
- Mächtige Abfragesprache mit einer Vielzahl an intuitiven URI Konventionen für Navigation, Filterung, Sortierung, Paging
- Uneingeschränkte Verwendung durch Microsoft Open Specification Promise (OSP) gewährleistet
- <http://www.odata.org/>



OData

• Die Entstehung von OData, V 4

- OData offiziell
- Open Data Philosophie
- SDK und Support
- OSP Lizenz



OData

- **4 Bausteine einer OData Implementierung**

- **OData Datenmodell**

- Generische Organisation und Beschreibung der Daten

- **OData Protokoll**

- Ermöglicht HTTP basierte RESTful CRUD Client – Server Interaktion auf Basis einer OData spezifischen Abfragesprache

- **OData Client Bibliotheken**

- Erleichtern die Erstellung von Software, die auf Daten mithilfe des OData Protokolls zugreifen.

- **OData Service**

- Ein abstrahiertes OData Datenmodell, ermöglicht es Daten in einem für den Client lesbaren Format darzustellen

OData

- **2 Bausteine eines OData Service**

- **Service Dokument**

- Darstellung sämtlicher Ressourcen eines Service, auf die über eine URI, einen Namen oder verschiedene Operationen zugegriffen werden kann.

- **Service Metadata Dokument**

- Darstellung sämtlicher Metadaten eines Services, wie Modell, Typen, Aktionen, Beziehungen und detaillierte Informationen über die Semantik des Modells

OData

- **Elemente eines OData Dokuments – EDM Konzept (1)**

- **Entity Type**

- Beschreibt eine Collection und hat einen Namen.

- **Entity**

- Instanzen eines Entity Types bestehend aus Schlüsselfeld(ern) und Eigenschaften z.B. Kunde, Mitarbeiter, ...

- **Entity Set**

- Entities sind in Entity Sets gruppiert z.B. Kunden als Entity Set des Entity Types Kunde.

OData

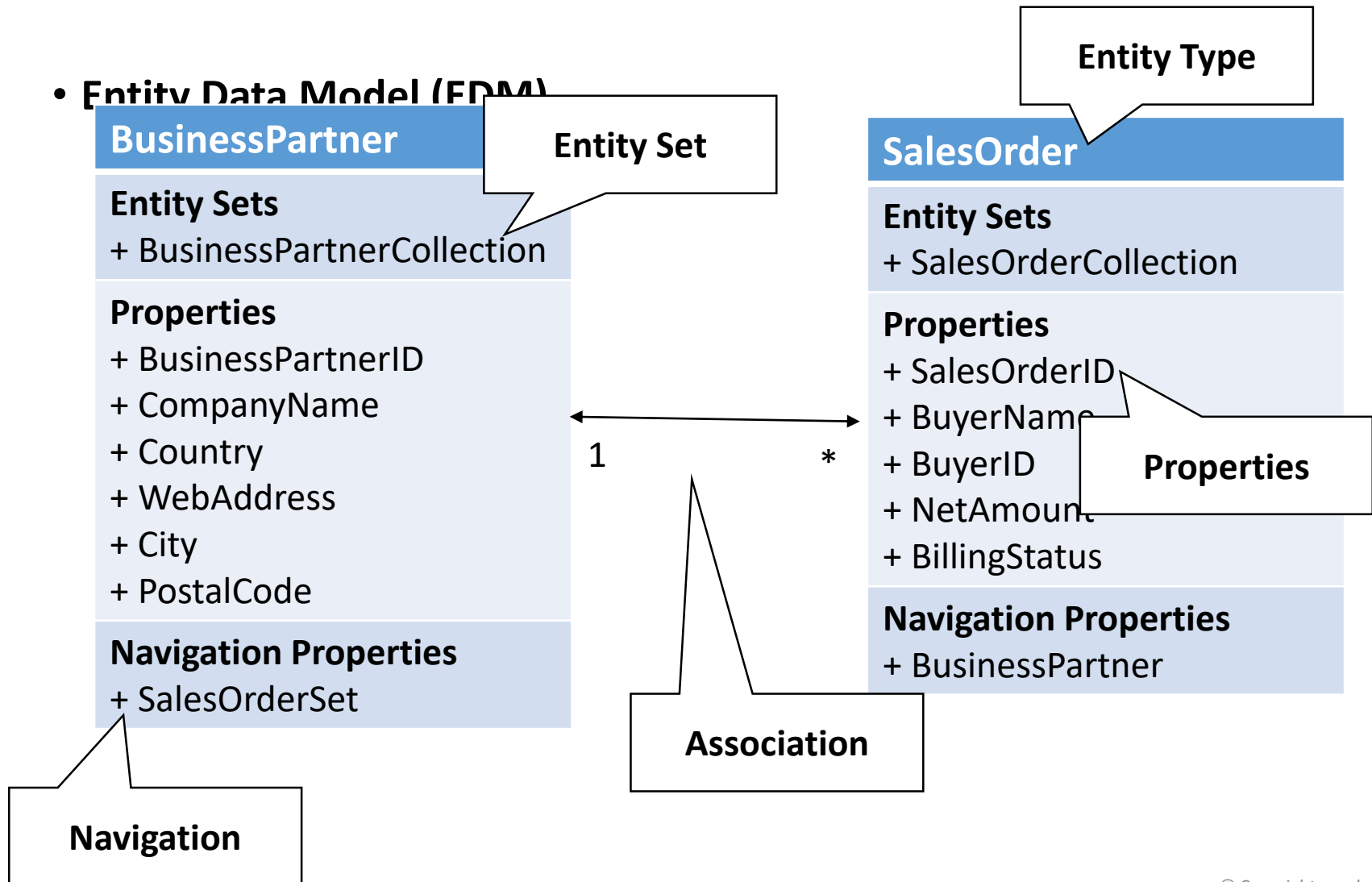
- **Elemente eines OData Dokuments – EDM Konzept (2)**

- **Property**
Einfaches Datenelement, Strukturierte Datenmenge oder Link auf andere Ressource.
- **Navigation property**
Zuständig für Links zwischen Entities.
- **Association**
Beschreibt die Beziehung zwischen Entity Types.



OData

• Entity Data Model (EDM)



OData

- **OData und ASP.NET Web API**

- Offenes und standardisiertes HTTP-basiertes Protokoll für die Interaktion mit REST-basierten Daten und Services
- Löst gängige Probleme (Query, Batching, Paging, Relationships, Metadata, etc.)
- Unterstützung von einer Vielzahl an Clients, Services und Frameworks



OData

- **OData in ASP.NET**
- Komponenten zur Erstellung von OData Services
 - Model Builder, Formatter, Path und Query Parser, Expression Generatoren
 - Individuell anpassbare Services
- OData Service in 3 Schritten
 - Model Definieren
 - OData Route hinzufügen
 - Entity Sets implementieren



OData

- http://localhost:1656/odata/\$metadata#Posts

GET

URL params

Send

Add to collection

Body

Headers (11)

STATUS 200 OK

TIME 19 ms

Pretty

Raw

Preview

JSON

XML

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <edmx:Edmx Version="1.0"
3   xmlns:edmx="http://schemas.microsoft.com/ado/2007/06/edmx">
4   <edmx:DataServices m:DataServiceVersion="3.0" m:MaxDataServiceVersion="3.0"
5     xmlns:m="http://schemas.microsoft.com/ado/2007/08/dataservices/metadata">
6     <Schema Namespace="ODataSamples.Controllers"
7       xmlns="http://schemas.microsoft.com/ado/2009/11/edm">
8       <EntityType Name="Post">
9         <key>
10          <PropertyRef Name="Id" />
11        </key>
12        <Property Name="Id" Type="Edm.Int32" Nullable="false" />
13        <Property Name="Title" Type="Edm.String" />
14        <Property Name="Author" Type="Edm.String" />
15        <Property Name="PublishingDate" Type="Edm.DateTime" Nullable="false" />
16      </EntityType>
17    </Schema>
18    <Schema Namespace="Default"
19      xmlns="http://schemas.microsoft.com/ado/2009/11/edm">
20      <EntityContainer Name="Container" m:IsDefaultEntityContainer="true">
21        <EntitySet Name="Posts" EntityType="ODataSamples.Controllers.Post" />
22      </EntityContainer>
23    </Schema>
24  </edmx:DataServices>
25 </edmx:Edmx>

```

OData

• Se| <http://localhost:1656/odata/Posts> GET [URL params](#)

[Send](#) [Add to collection](#)

Body Headers (11) STATUS 200 OK TIME 41 ms

Pretty Raw Preview   JSON XML

```

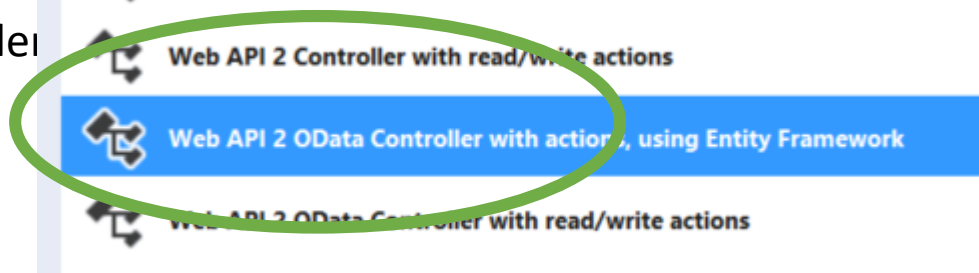
1 {
2   "odata.metadata": "http://localhost:1656/odata/$metadata#Posts",
3   "value": [
4     {
5       "Id": 1,
6       "Title": "post1",
7       "Author": "ema",
8       "PublishingDate": "2013-04-05T00:00:00+02:00"
9     },
10    {
11      "Id": 2,
12      "Title": "post2",
13      "Author": "ale",
14      "PublishingDate": "2013-04-06T00:00:00+02:00"
15    },
16    {
17      "Id": 3,
18      "Title": "post3",
19      "Author": "vale",
20      "PublishingDate": "2013-04-03T00:00:00+02:00"
21    },
22    {
23      "Id": 4,
24      "Title": "post4",
25      "Author": "lory",
26      "PublishingDate": "2013-04-06T00:00:00+02:00"
27    },
28    {
29      "Id": 5,
30      "Title": "post5",
31      "Author": "tara"

```

OData

- **OData API Controller**

- Implementations



- ASP.NET Web API unterstützt OData
- Voller Support ab Visual Studio 2012 Update 2, oder
- Microsoft ASP.NET Web API OData NuGet Packet

OData

- **Route** in App_Start/WebApiConfig.cs

```
// OData
```

```
ODataModelBuilder builder = new  
ODataConventionModelBuilder();  
builder.EntitySet<Categories>("Categories1");
```

```
Microsoft.Data.Edm.IEdmModel model = builder.GetEdmModel();  
config.Routes.MapODataRoute(  
    routeName: "ODataRoute",  
    routePrefix: "odata", // null  
    model: model);
```

OData

- **Grundlagen OData Operationen(1)**

- In OData API Controller umsetzen!
- Komplette Übersicht möglicher Operationen in OData V. 4:
 - <http://docs.oasis-open.org/odata/odata/v4.0/odata-v4.0-part1-protocol.html>
- **Metadata:**
Aufruf Service Meta Dokument
`/odata/$metadata`
- **OrderBy:**
Sortierung des Dokuments, mehrere Attribute möglich
`/odata/Posts?$orderby=Author desc`
`/odata/Posts?$orderby=Author,PublishingDate`

OData

- **Grundlagen OData Operationen(2)**

- **Top:**

- Anzahl der zurückgegeben Elemente

- `/odata/Posts?$top=2`

- **Inlinecount:**

- Anzahl der Items in der Collection

- `/odata/Posts?$inlinecount=allpages`

- **Skip:**

- Überspringen der ersten n Einträge

- `/odata/Posts?$skip=2`

OData

- **Grundlagen OData Operationen(3)**

- **Filter:**

- `/odata/Posts?$filter=Author%20eq%20'ema'`

- `/odata/Posts?$filter=Author%20eq%20'ema'%20and%20Title%20eq%20'post2'`

Operator	Beschreibung	C# Equivalent
eq	Equal	==
ne	Not equal	!=
gt	Greater than	>
ge	Greater than or equal	>=
lt	Less than	<
le	Less than or equal	<=
and	And	&&
or	Or	

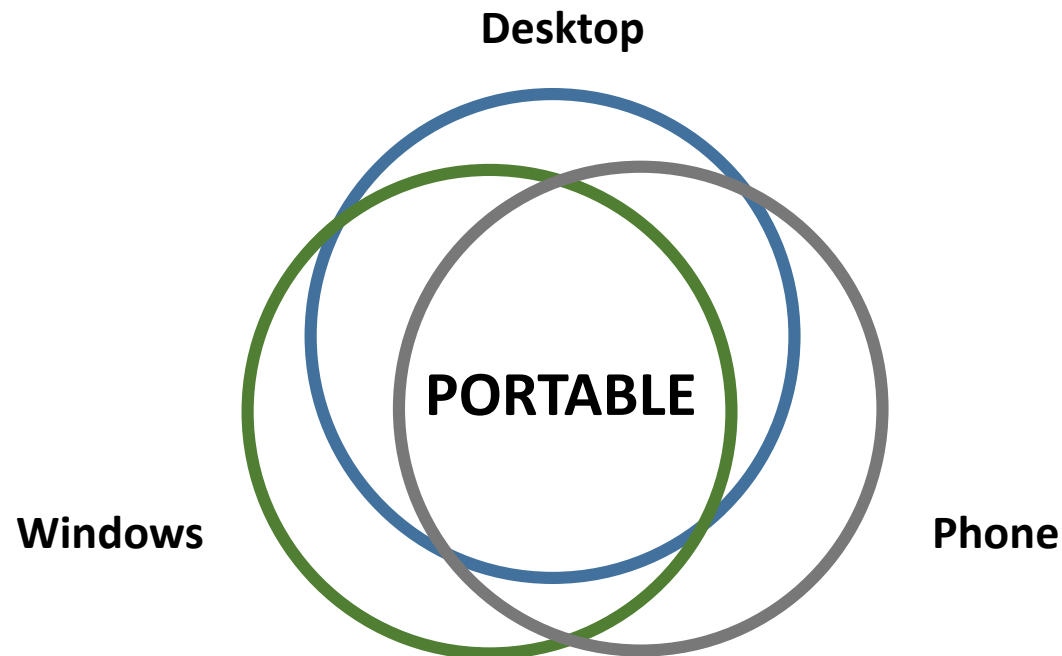
Clients

Für die Web API 2.0



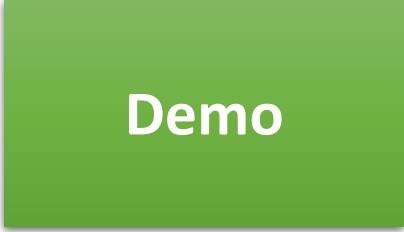
Clients

- Logik in „Portable Class Libraries“



Clients

- **JavaScript-Anwendung**
 - AJAX Call
 - `XMLHttpRequest()` ;
- **WPF-Anwendung**
 - Data Binding



Demo

